

University of East London

UNIVERSITY OF EAST LONDON

School of Architecture, Computing and Engineering

MSc DATA SCIENCE

DISSERTATION

A Comparative Study of Machine Learning Models for Predicting the Remaining Useful Life (RUL) of Aircraft Turbofan Engines

Submitted by	Victor Osasumwen Ogedegbe
Programme	MSc Data Science
Submission Date	August 2026

Submitted in partial fulfilment of the requirements
for MSc Data Science

ACKNOWLEDGEMENT

I give thanks to Almighty God for His grace, strength, and guidance throughout my MSc Data Science programme and the completion of this dissertation.

I am deeply grateful to my parents for their sacrifices, constant support, encouragement, inspiration, and unwavering belief in me throughout my academic journey. Their guidance has been a major source of motivation and has helped me reach this stage.

I sincerely appreciate my project supervisor, Mr Joseph Annan, for his guidance, constructive feedback, and continuous encouragement throughout this project. His support consistently encouraged me to explore my research in greater depth, challenged me to improve my work, and contributed greatly to the overall quality of this dissertation.

My sincere thanks also go to Dr Yang Li, Mr Modi, and the other lecturers and academic staff at the University of East London for the knowledge, guidance, and support they provided throughout my programme.

I also acknowledge the NASA Prognostics Center of Excellence for making the C-MAPSS dataset publicly available, thereby supporting the research presented in this dissertation.

Finally, I am grateful to my siblings, friends, and colleagues for their encouragement and support in various ways throughout this journey. I am also proud of myself for believing in my abilities, remaining determined, and continuing to move forward despite the challenges along the way.

ABSTRACT

Aircraft turbofan engines gradually deteriorate as they operate. Predicting their Remaining Useful Life (RUL), meaning the number of operating cycles left before failure, can help maintenance teams plan maintenance before failure occurs.

This study compares five prediction models for predicting the RUL of aircraft turbofan engines using the NASA C-MAPSS FD001 dataset.

The FD001 dataset contains data from 100 simulated engines for training and 100 separate simulated engines for testing. All engines operate under one operating condition and experience one fault type. After removing variables with little or no variation, 17 input variables were retained. RUL values were calculated from the engine data and capped at 125 cycles.

A baseline model and four machine learning models were evaluated: a Mean RUL Baseline, Linear Regression, Random Forest, Support Vector Regression (SVR) and Long Short-Term Memory (LSTM). The training and validation data were separated by engine to reduce data leakage. Cross-validation was used for the baseline and classical machine learning models. Random Forest and SVR were also optimized using hyperparameter tuning, while the LSTM used a fixed architecture with early stopping.

The models were evaluated on the official NASA test set using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), R^2 and the NASA PHM Score. The LSTM produced the best overall results, with an MAE of 10.42 cycles, an RMSE of 14.60 cycles, an R^2 of 0.867 and a NASA PHM Score of 314. Random Forest was the best classical machine learning model, achieving an MAE of 12.40 cycles, an RMSE of 16.70 cycles, an R^2 of 0.826 and a NASA PHM Score of 491.

Overall, the LSTM predicted RUL more accurately than the other models. However, Random Forest provided a strong alternative because it required less computational effort and was easier to interpret. Since the FD001 dataset is simulated and represents only one operating condition and one fault type, the findings should not be applied directly to real aircraft engines without further testing.

Keywords: Remaining Useful Life (RUL), predictive maintenance, aircraft turbofan engines, NASA C-MAPSS, machine learning, Long Short-Term Memory (LSTM), prognostics.

TABLE OF CONTENTS

1 Introduction....12
1.1 Summary....12
1.2 Background....12
1.3 Aim....14
1.4 Research Objectives....14
1.5 Research Questions....15
1.6 Data Source....15
2 Literature Review....16
2.1 Introduction....16
2.2 Background to Aircraft Turbofan RUL Prediction....16
2.2.1 Aircraft Turbofan Engines....16
2.2.2 Predictive Maintenance....18
2.2.3 Remaining Useful Life (RUL)....19
2.3 Machine Learning....21
2.3.1 Types of Machine Learning....22
2.3.1.1 Supervised Learning....23
2.3.1.2 Unsupervised Learning....24
2.3.1.3 Semi-supervised Learning....25
2.3.1.4 Reinforcement Learning....25
2.4 Machine Learning for RUL Prediction....26
2.4.1 Classical Machine Learning Models....28
2.4.1.1 Linear Regression....29
2.4.1.2 Support Vector Regression (SVR)....30
2.4.1.3 Random Forest....31
2.4.1.4 Extreme Gradient Boosting (XGBoost)....32
2.4.2 Non classical Models (Deep learning)....33
2.4.2.1 Recurrent Neural Networks (RNN)....34
2.4.2.2 Long Short-Term Memory (LSTM)....34
2.4.2.3 Transformer....35
2.5 Comparative Review of Machine Learning Studies for Turbofan RUL Prediction....35
2.6 Challenges and Limitations....38
2.7 Chapter Summary....39

3 Methodology....41
3.1 Introduction....41
3.2 Overall Research Workflow....41
3.3 Research Design....42
3.4 Data Source and Dataset Description....43
3.5 Data Processing....44
3.5.1 Data Quality Checks....44
3.5.2 RUL Target Construction....44
3.5.3 Feature Scaling (Standardization)....45
3.5.4 Feature Selection....45
3.6 Exploratory Data Analysis....46
3.6.1 Distribution of the Target Variable....46
3.6.2 Normality Assessment....46
3.6.3 Sensor Trend Analysis....47
3.6.4 Correlation Analysis....47
3.7 Machine Learning Models....48
3.7.1 Mean RUL Baseline....48
3.7.2 Linear Regression....48
3.7.3 Random Forest....49
3.7.4 Support Vector Regression (SVR)....49
3.7.5 Long Short-Term Memory (LSTM)....49
3.8 Data Splitting and Validation Strategy....50
3.9 Hyperparameter Optimization....51
3.10 Model Evaluation....52
3.11 Performance Evaluation Metrics....52
3.12 Model Interpretation and Diagnostic Analysis....53
3.13 Chapter Summary....54
4 Implementation....55
4.1 Introduction....55
4.2 Overall Research Workflow....55
4.3 Script Initialization....56
4.4 Data Loading....56
4.5 Data Processing....57
4.5.1 Data Quality Checks....57

4.5.2 RUL Target Construction....57
4.5.3 Feature Scaling (Standardization)....58
4.5.4 Feature Selection....58
4.6 Exploratory Data Analysis....59
4.6.1 Distribution of the Target Variable....59
4.6.2 Normality Assessment....60
4.6.3 Sensor Trend Analysis....60
4.6.4 Correlation Analysis....61
4.7 Machine Learning Models....61
4.7.1 Mean RUL Baseline....62
4.7.2 Linear Regression....62
4.7.3 Random Forest....62
4.7.4 Support Vector Regression (SVR)....62
4.7.5 Long Short-Term Memory (LSTM)....63
4.8 Data Splitting and Cross-Validation Strategy....64
4.9 Hyperparameter Optimization....65
4.10 Final Model Retraining....66
4.11 Model Evaluation....66
4.12 Performance Evaluation Metrics....67
4.13 Model Interpretation and Diagnostic Analysis....67
4.14 Chapter Summary....68
5 Results and Analysis....69
5.1 Introduction....69
5.2 Data Preparation Results....69
5.3 Exploratory Data Analysis Results....70
5.3.1 Distribution of the Target Variable....71
5.3.2 Normality Assessment....72
5.3.3 Sensor Trend Analysis....73
5.3.4 Correlation Analysis....74
5.4 Model Development....76
5.5 Hyperparameter Optimization Results....76
5.6 Validation and Cross-Validation Results....77
5.6.1 Held-Out Validation....77
5.6.2 Overfitting Check....78

5.6.3 Five-Fold Grouped Cross-Validation....79
5.6.4 LSTM Training....81
5.7 Official NASA Test Set Results....82
5.7.1 Mean Baseline....84
5.7.2 Linear Regression....85
5.7.3 Random Forest....86
5.7.4 Support Vector Regression (SVR)....87
5.7.5 LSTM....88
5.8 Model Interpretation Results....89
5.9 Critical Evaluation of the Optimum Model....90
5.10 Chapter Summary....91
6 Critical Evaluation, Discussion and Conclusion....93
6.1 Introduction....93
6.2 Critical Evaluation of the Research Process....93
6.2.1 Critical Evaluation of the Research Design....93
6.2.2 Critical Evaluation of the Methodological Implementation....94
6.2.3 Critical Evaluation of Model Development and Evaluation Validity....94
6.3 Dissertation Diary and Critical Reflection....95
6.4 Discussion and Synthesis of the Findings....96
6.5 Conclusions in Relation to the Aim and Research Questions....99
6.6 Limitations....99
6.7 Future Direction....100
6.7.1 Practical Recommendations....100
6.7.2 Academic Contribution....101
6.7.3 Future Research....101
6.8 Overall Conclusion....102
References....103
APPENDICES....107

TABLE OF FIGURES

Figure 2.1: Cross-sectional diagram of a two-spool turbofan engine showing the major components and airflow. **Source:** Wheeler (2009), Wikimedia Commons.....17
Figure 2.2: Progression of maintenance strategies, from reactive corrective maintenance to proactive RUL prediction.....18
Figure 2.3: Conceptual illustration of Remaining Useful Life. RUL is the interval between the current time and the point at which the health indicator reaches the failure threshold.20
Figure 2.4: Relationship between AI and machine learning (adapted from Goodfellow, Bengio and Courville, 2016).....21
Figure 2.5: Types of machine learning (Adapted from Mitchell, 1997; Goodfellow, Bengio and Courville, 2016).....22
Figure 2.6: Comparison of classical machine learning and deep learning approaches for RUL prediction. Source: Author's illustration based on LeCun, Bengio and Hinton (2015) and Goodfellow, Bengio and Courville (2016).....28
Figure 2.7: Illustration of a linear regression best-fit line, showing the relationship between predictor (x) and target (y) variables (author's illustration).....30
Figure 2.8: Support vector regression illustrating the hyperplane, maximum margin, and support vectors (Van Otten, 2024).....31
Figure 2.9: Illustration of the random forest algorithm (Jagannath, 2017).....32
Figure 2.10: Architecture of a Long Short-Term Memory (LSTM) unit showing the memory cell, input gate, forget gate and output gate. Source: MathWorks (n.d).....34
Figure 3.1: Overall research workflow.....42
Figure 4.1: The eleven main steps of rul_prediction.py, in the order they run.....55
Figure 4.2: Script initialization showing the global configuration, reproducibility settings and automatic creation of the output directories.....56
Figure 4.3: Loading the three FD001 files and checking their structure.....57
Figure 4.4: The function used to generate and save the missingness maps for the training and test data.....57
Figure 4.5: Implementation of the Remaining Useful Life (RUL) target construction.....58
Figure 4.6: The scaling pipelines used for Linear Regression and SVR.....58
Figure 4.7: Implementation of the sensor and operational setting variability checks.....59
Figure 4.8: Standardizing the predictors before any exploratory analysis is produced.....59
Figure 4.9: Generating the RUL histogram and boxplot.....60
Figure 4.10: Implementation of the statistical normality tests.....60
Figure 4.11: Implementation of the sensor trend analysis.....61
Figure 4.12: Implementation of the Pearson correlation heatmap.....61
Figure 4.13: Implementation of the Mean RUL Baseline prediction.....62
Figure 4.14: Implementation of the Linear Regression pipeline.....62
Figure 4.15: Implementation of the Random Forest model definition.....62
Figure 4.16: Implementation of the Support Vector Regression (SVR) pipeline.....63
Figure 4.17: Implementation of the sliding-window sequence generation for the LSTM.....63
Figure 4.18: Implementation of the LSTM network architecture.....64
Figure 4.19: Implementation of the engine-level training and validation split.....64
Figure 4.20: Implementation of the 5-fold GroupKFold cross-validation.....65
Figure 4.21: Implementation of hyperparameter optimization using RandomizedSearchCV.....65
Figure 4.22: Implementation of the final model retraining.....66
Figure 4.23: Preparing the official test set: last cycle per engine and capped RUL.....66

Figure 4.24: Implementation of the LSTM test sequence generation.....67
Figure 4.25: Implementation of the model evaluation metrics, including the NASA PHM Score. 67
Figure 4.26: Implementation of impurity-based feature importance for the Random Forest model. 68
Figure 4.27: Implementation of the permutation importance analysis.....68
Figure 5.1 Data quality checks. a: terminal output showing engine counts, RUL entries, missing values and duplicate records. b: missingness map for the training dataset.....69
Figure 5.2: Terminal output showing the results of the near-constant predictor removal process.....70
Figure 5.3: Distribution of the capped RUL target across the training dataset.....71
Figure 5.4: Quantile–quantile (QQ) plot of the capped RUL target.....72
Figure 5.5: Sensor 2 readings over the life of Engine 1.....73
Figure 5.6: Correlation heatmap of all 17 predictors and RUL.....74
Figure 5.7: Pearson correlation of each feature with RUL, ranked.....75
Figure 5.8: RMSE across the five grouped cross-validation folds.....79
Figure 5.9: LSTM training and validation loss across epochs.....81
Figure 5.10: Actual versus predicted RUL — Mean Baseline.....84
Figure 5.11: Residual distribution — Mean Baseline.....84
Figure 5.12: Actual versus predicted RUL for the Linear Regression model.....85
Figure 5.13: Residual distribution for the Linear Regression model.....85
Figure 5.14: Actual versus predicted RUL — Random Forest.....86
Figure 5.15: Residual distribution — Random Forest.....86
Figure 5.16: Actual versus predicted RUL — SVR.....87
Figure 5.17: Residual distribution — SVR.....87
Figure 5.18: Actual versus predicted RUL — LSTM.....88
Figure 5.19: Residual distribution — LSTM.....88
Figure 5.20: Random Forest feature importance — impurity-based (left) and permutation (right) 89

LIST OF TABLES

Table 2.1:Comparison of the main approaches to RUL prediction.....19
Table 2.2:Summary of representative studies on machine learning models for turbofan Remaining Useful Life prediction, showing the datasets, models, evaluation metrics, key findings and limitations.....36
Table 3.1: Summary of the C-MAPSS FD001 dataset used in this study.....44
Table 3.2: Performance evaluation metrics used in this study.....53
Table 5.1: Normality test results for the capped RUL target.....72
Table 5.2: Best hyperparameters found for Random Forest and SVR.....76
Table 5.3: Performance of the five models on the 20 held-out validation engines.....77
Table 5.4: Training versus validation R^2 for each classical model.....78
Table 5.5: Average cross-validation performance across all five folds.....80
Table 5.6: Final performance of all five models on the official NASA test set (RUL capped at 125).....82
Table 5.7: Training time comparison of the machine learning models (Times were measured on the same hardware and software environment and are intended for relative comparison within this study, not as universal benchmarks).....83

TABLE 5.8: SUMMARY COMPARISON OF MODEL ACCURACY, COMPUTATIONAL COST AND INTERPRETABILITY ON THE OFFICIAL NASA TEST SET.....90
TABLE 6.1: SUMMARY OF RESEARCH QUESTIONS, FINDINGS AND CONCLUSIONS.....99

1 Introduction

1.1 Summary

This study investigates the use of machine learning models for predicting the Remaining Useful Life (RUL) of aircraft turbofan engines. RUL refers to the estimated number of operating cycles an engine has left before failure.

Predicting RUL is an important part of predictive maintenance because it can help maintenance teams plan repairs, improve reliability and reduce the risk of unexpected engine failure.

The study begins by reviewing existing research on predictive maintenance, RUL prediction and the use of machine learning to estimate the remaining life of turbofan engines. It then develops a methodology for preparing the data and comparing selected machine learning models under a consistent experimental framework.

The selected models are implemented, trained and evaluated using the same dataset and common regression performance measures to support a fair comparison. Their performance is also examined in terms of prediction accuracy, computational cost and interpretability.

Finally, the findings are critically analyzed to identify the strengths, limitations and suitability of the selected models for RUL prediction. The study concludes by discussing the limitations of the research and providing recommendations for future work in machine-learning-based predictive maintenance.

1.2 Background

Maintenance is important for keeping engineering systems safe, reliable and available for use. Corrective maintenance involves repairing equipment after failure, while preventive maintenance involves servicing equipment at fixed intervals. Both approaches can lead to unexpected failures or unnecessary maintenance when the actual condition of the equipment is not considered.

Predictive maintenance addresses this issue by using condition and operational data to estimate when maintenance may be required, helping to reduce downtime and improve reliability (Jardine, Lin and Banjevic, 2006).

A key part of predictive maintenance is Remaining Useful Life (RUL) prediction. RUL is the estimated amount of operating time or number of cycles remaining before a component reaches the end of its useful life. Accurate RUL prediction can support maintenance planning before failure occurs. However, degradation is affected by operating conditions, environmental factors and differences in how equipment deteriorates over time, making RUL prediction challenging (Si et al., 2011).

Aircraft turbofan engines are an important application of RUL prediction because they operate under demanding conditions and produce sensor measurements throughout their operation. These measurements can indicate changes in engine condition and degradation, making them suitable for data-driven prediction.

Machine learning is widely used for RUL prediction because it can learn relationships between sensor data and engine degradation. Models such as Random Forest, Support Vector Regression and Long Short-Term Memory (LSTM) have been applied to turbofan engine RUL prediction. However, their performance can vary depending on the data, preprocessing methods and modelling approaches used (Zheng et al., 2017; Wang et al., 2023; Wu et al., 2024).

The NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) dataset is widely used for turbofan engine RUL prediction. It contains multivariate time-series data generated from simulated turbofan engines operating under different conditions and fault modes (Saxena and Goebel, 2008). Its widespread use provides a common benchmark for developing, evaluating and comparing RUL prediction models.

Although many machine learning models have been used for turbofan engine RUL prediction, differences in preprocessing, model settings and evaluation procedures make direct comparison difficult. This creates uncertainty about which models provide the strongest performance when evaluated under a common experimental framework.

This study therefore compares selected machine learning models for predicting the RUL of aircraft turbofan engines using the NASA C-MAPSS dataset within a consistent experimental framework.

1.3 Aim

The aim of this study is to evaluate and compare selected machine learning models for predicting the Remaining Useful Life (RUL) of aircraft turbofan engines using the NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) dataset.

1.4 Research Objectives

The objectives of this study are to:

- Review the existing literature on predictive maintenance, Remaining Useful Life (RUL) prediction and the application of machine learning to aircraft turbofan engine RUL prediction.
- Design a methodology for preprocessing the NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) dataset and preparing the data for model development.
- Develop and implement selected machine learning models for predicting the Remaining Useful Life of aircraft turbofan engines.
- Evaluate and compare the performance of the selected machine learning models using appropriate regression evaluation metrics.
- Critically analyze the findings by considering the strengths, limitations and suitability of the selected models for RUL prediction and provide recommendations for future research.

1.5 Research Questions

This study addresses the following research questions:

- What does the existing literature show about the use of machine learning for predicting the Remaining Useful Life (RUL) of aircraft turbofan engines?
- What data-preprocessing and modelling methods are suitable for predicting RUL using the NASA C-MAPSS FD001 dataset?
- How accurately can the selected machine learning models predict the RUL of aircraft turbofan engines?
- Which of the selected machine learning models provides the best predictive performance on the same test dataset?
- What are the main strengths, limitations and practical trade-offs of the selected machine learning models for RUL prediction?

1.6 Data Source

This study uses the FD001 subset of the NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) dataset as the primary data source. Developed by the NASA Prognostics Center of Excellence, the dataset contains multivariate time-series sensor data generated from simulated turbofan engines. The FD001 subset was selected because it is a widely used benchmark for Remaining Useful Life (RUL) prediction and represents a single operating condition and a single fault mode. Its widespread use in predictive maintenance research supports comparison between the findings of this study and previous research.

2 Literature Review

2.1 Introduction

This chapter reviews the literature on machine learning for predicting the Remaining Useful Life (RUL) of aircraft turbofan engines. RUL prediction is an important part of predictive maintenance because it can support maintenance planning before engine failure occurs.

Although many machine learning models have been applied to this problem, their reported performance differs considerably across studies. These differences are influenced not only by the models themselves, but also by data preprocessing, feature selection, model configuration and evaluation procedures. As a result, reported results cannot always be compared directly.

The chapter first reviews the role of aircraft turbofan engines, predictive maintenance and RUL prediction before considering the machine learning approaches used in this area. Attention is given to classical machine learning and deep learning models, including their strengths, limitations and suitability for RUL prediction.

Previous studies are then compared to identify common findings, methodological differences and areas of disagreement. The chapter concludes by examining the main limitations in the existing literature addressed by this study.

2.2 Background to Aircraft Turbofan RUL Prediction

2.2.1 Aircraft Turbofan Engines

Turbofan engines are the primary propulsion system used in modern commercial and military aircraft because of their high thrust, fuel efficiency and operational reliability. Figure 2.1 shows the main components of a typical two-spool turbofan engine. During operation, these components are exposed to varying operating conditions, thermal stresses and mechanical wear, resulting in gradual performance degradation over time.

Unlike systems that fail suddenly, turbofan engines typically degrade progressively, allowing changes in engine health to be monitored through onboard sensor measurements (Rolls-Royce, 2015).

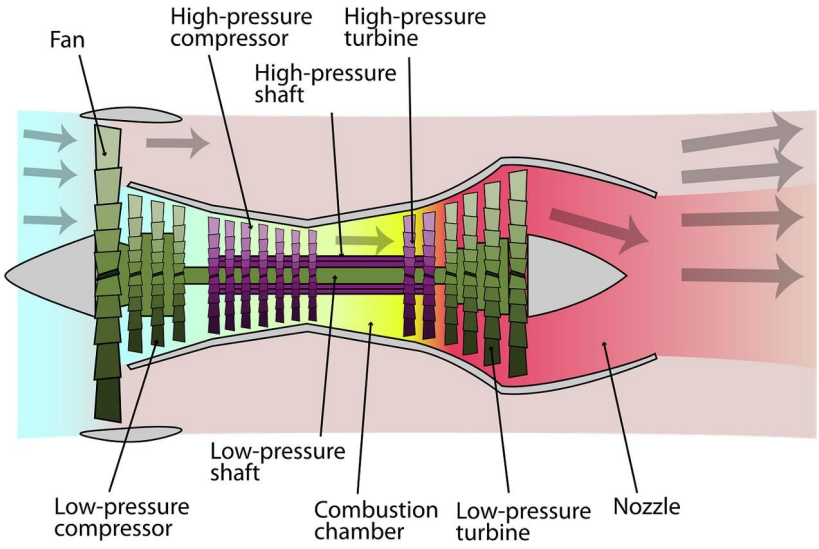


Figure Literature Review.1: Cross-sectional diagram of a two-spool turbofan engine showing the major components and airflow. ***Source:*** *Wheeler (2009), Wikimedia Commons.*

Modern turbofan engines generate large volumes of sensor data, including measurements of temperature, pressure, rotational speed and fuel flow. These data support health monitoring and fault prediction by providing information about the engine's condition. However, degradation is influenced by operating conditions, component wear, measurement uncertainty and sensor noise, making accurate prediction difficult even when large amounts of sensor data are available (Lei et al., 2018).

To address this challenge, researchers estimate the Remaining Useful Life (RUL), defined as the operating cycles or time remaining before an asset reaches a predefined failure threshold (Si et al., 2011). Accurate RUL prediction enables maintenance to be scheduled according to the engine's condition rather than fixed service intervals, reducing unnecessary maintenance while lowering the risk of unexpected failures. As a result, RUL prediction has become a central objective of predictive maintenance research and has driven the widespread application of machine learning techniques for turbofan engine RUL prediction.

2.2.2 Predictive Maintenance

Maintenance is essential for ensuring the safety, reliability and operational availability of aircraft engines. Traditional maintenance strategies include corrective and preventive maintenance, but both have limitations. Corrective maintenance is performed only after a component has failed, often leading to unplanned downtime, higher repair costs and operational disruption. Preventive maintenance schedules servicing at fixed intervals regardless of the equipment's condition. Although this reduces the likelihood of unexpected failures, it can increase costs through unnecessary maintenance and the early replacement of serviceable components (Jardine, Lin and Banjevic, 2006).

Predictive maintenance is a maintenance strategy that uses operational and condition-monitoring data to assess equipment health and predict when maintenance should be performed. Instead of following fixed service intervals, maintenance decisions are based on the actual condition of the equipment. This approach aims to maximize component utilization, reduce unnecessary servicing and minimize unexpected failures.

However, its effectiveness depends on the availability of reliable sensor data and the ability to accurately predict future degradation. Consequently, Remaining Useful Life (RUL) estimation has become a key component of predictive maintenance because it estimates how long an asset can continue operating before reaching failure, enabling maintenance to be planned based on predicted asset health rather than elapsed operating time (Lei et al., 2018).

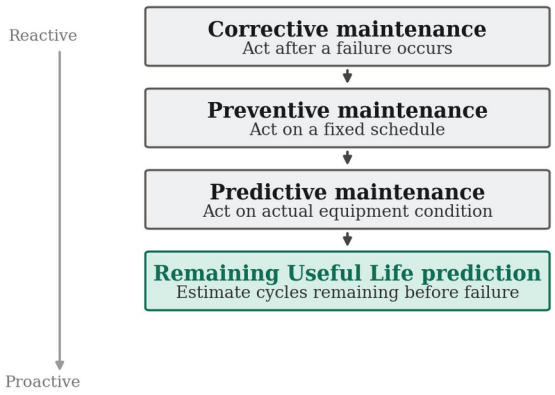


Figure Literature Review.2: Progression of maintenance strategies, from reactive corrective maintenance to proactive RUL prediction.

2.2.3 Remaining Useful Life (RUL)

Remaining Useful Life (RUL) is the estimated time or number of operating cycles before a component or system reaches a predefined failure threshold (Si et al., 2011). Within predictive maintenance, RUL supports maintenance planning by estimating how long equipment can continue operating before maintenance is required. Accurate RUL estimation helps reduce unexpected failures and unnecessary maintenance, whereas inaccurate estimates may increase maintenance costs or the risk of in-service failure.

Estimating RUL is challenging because equipment degradation rarely follows a consistent pattern. Changes in operating conditions, environmental factors, sensor noise and differences between individual engines introduce uncertainty into the degradation process, making accurate prediction difficult (Lei et al., 2018). As a result, methods based on fixed degradation assumptions often struggle to perform reliably across different operating conditions.

To address these challenges, researchers generally use three approaches for RUL prediction: physics-based, statistical and data-driven methods.

Physics-based and statistical approaches can provide useful insight into degradation, but they depend on detailed system knowledge or modelling assumptions.

Data-driven methods instead learn degradation patterns directly from historical sensor data, making them well suited to complex turbofan engine data.

Table 2.1 summarizes the main differences between these approaches. Because this study focuses on machine learning, the remainder of the review concentrates on data-driven methods.

Table Literature Review.1:Comparison of the main approaches to RUL prediction.

Approach	Principle	Advantages	Limitations	Typical applications
Physics-based	Uses physical degradation models	High interpretability	Requires detailed system knowledge	Aerospace, mechanical systems
Statistical	Uses probabilistic degradation modelling	Handles uncertainty	Performance depends on assumptions	Reliability engineering
Data-driven	Learns degradation directly from historical sensor data	High predictive accuracy and scalability	Requires sufficient labelled data	Predictive maintenance and PHM

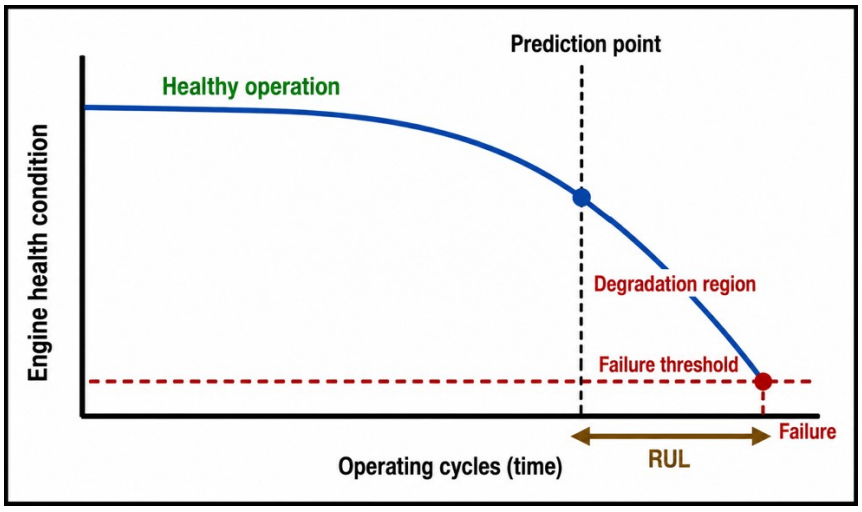


Figure Literature Review.3: Conceptual illustration of Remaining Useful Life. RUL is the interval between the current time and the point at which the health indicator reaches the failure threshold.

Figure 2.3 shows RUL as the period between the current prediction point and the point at which the engine reaches the failure threshold. This illustrates how changes in engine condition over time can be used to estimate the remaining operating life.

2.3 Machine Learning

Machine learning (ML) is a branch of artificial intelligence (AI) in which algorithms learn patterns from data and use these patterns to make predictions or decisions on new observations (Mitchell, 1997). Unlike conventional rule-based programming, where relationships must be specified in advance, machine learning allows relationships to be estimated from historical data. This is particularly useful where the relationship between the predictors and the required output is complex or difficult to express through fixed mathematical rules.

This characteristic makes machine learning relevant to Remaining Useful Life (RUL) prediction. Turbofan engines generate multivariate sensor measurements over successive operating cycles, and these measurements may contain information about changes in engine condition and degradation.

Machine learning provides a data-driven means of relating these observations to remaining operating life, including nonlinear relationships that may be difficult to capture using simpler fixed models (Goodfellow, Bengio and Courville, 2016; Lei et al., 2018). However, the usefulness of machine learning depends not only on the algorithm selected but also on the availability of suitable data, the definition of the prediction target and the way the model is evaluated.

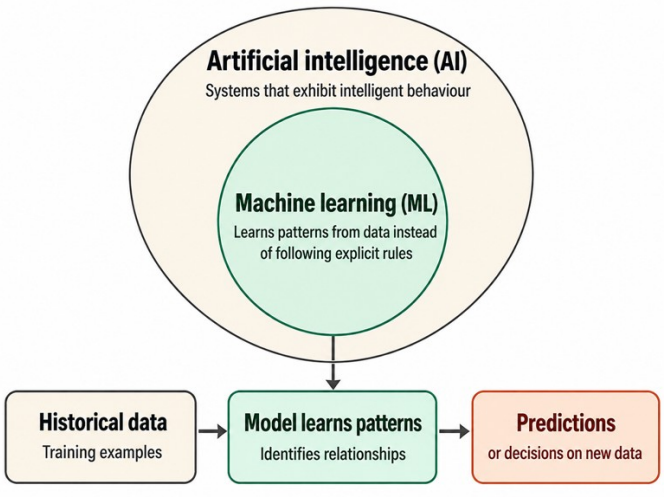


Figure Introduction.4: Relationship between AI and machine learning (adapted from Goodfellow, Bengio and Courville, 2016)

Machine learning contains several learning paradigms that differ in the information available during training and the objective of the learning process. Understanding these differences is important in this study because the form of the RUL prediction problem determines which learning paradigm is most appropriate.

2.3.1 Types of Machine Learning

Machine learning algorithms can be broadly classified into four main learning paradigms: supervised learning, unsupervised learning, reinforcement learning and semi-supervised learning.

A learning paradigm refers to the way a machine learning algorithm learns from data and improves its performance. These paradigms differ in the way they learn from data, particularly in terms of the availability of labelled data and the learning process adopted by the algorithm (Mitchell, 1997; Goodfellow, Bengio and Courville, 2016).

Each approach has been developed to address different types of problems and has found applications across a wide range of domains.

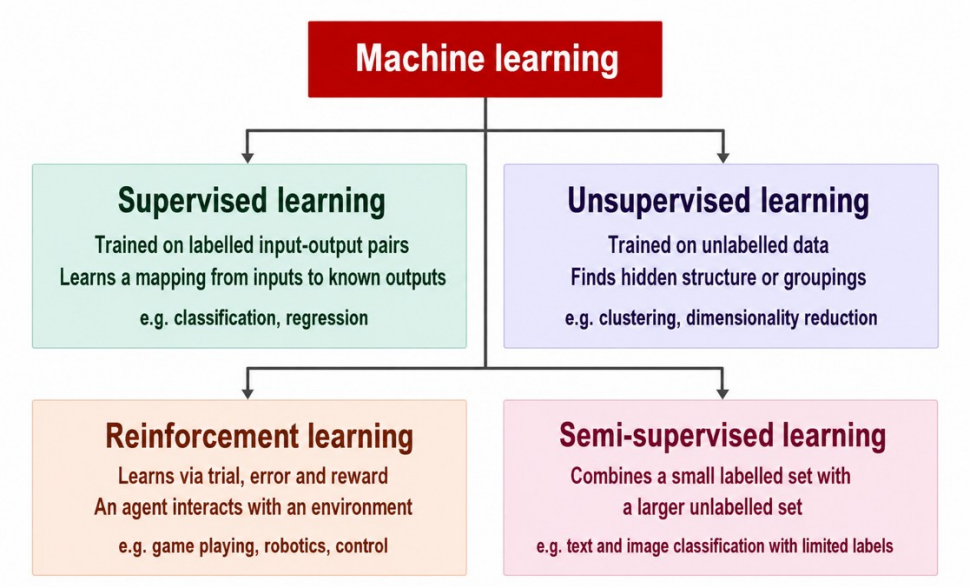


Figure Literature Review.5: Types of machine learning (Adapted from Mitchell, 1997; Goodfellow, Bengio and Courville, 2016)

The following sections review these four learning paradigms and highlight their characteristics, common algorithms and areas of application.

2.3.1.1 Supervised Learning

Supervised learning is the most widely used machine learning paradigm and involves training a model using labelled data, where each input is paired with a known output or target value (Mitchell, 1997). These known outputs act as the correct answers, allowing the algorithm to learn the relationship between the input variables and the target variable during training. Once trained, the model applies this learned relationship to make predictions on new, unseen data. The term supervised refers to the use of these labelled examples to guide the learning process and improve predictive performance (Goodfellow, Bengio and Courville, 2016).

Supervised learning is commonly applied to two main types of problems: classification and regression.

Classification predicts discrete or categorical outcomes, such as whether an email is spam or not spam, whereas regression predicts continuous numerical values, such as house prices, stock prices or temperature (Bishop, 2006). Consequently, a wide range of supervised learning algorithms has been developed, including Linear Regression, Logistic Regression, Decision Trees, Random Forest, Support Vector Machines (SVM), Artificial Neural Networks and k-Nearest Neighbours (k-NN). The choice of algorithm depends on the characteristics of the dataset, the complexity of the problem and the required level of predictive performance (Goodfellow, Bengio and Courville, 2016).

Because supervised learning learns from historical labelled data, it has become one of the most widely adopted approaches across fields such as healthcare, finance, cybersecurity, manufacturing and engineering. Its ability to model relationships between input variables and known outputs makes it particularly suitable for prediction tasks where historical observations are available. For turbofan Remaining Useful Life (RUL) prediction, historical sensor measurements are paired with known RUL values, allowing supervised learning algorithms to learn the relationship between engine condition and the remaining operating life.

Consequently, supervised learning is the main learning paradigm adopted in this dissertation, with the four machine learning models formulated as supervised regression models alongside a non-learning baseline for comparison.

2.3.1.2 Unsupervised Learning

Unlike supervised learning, unsupervised learning is a machine learning paradigm that uses unlabelled data, meaning the training data do not contain known output labels or target values (Goodfellow, Bengio and Courville, 2016). Instead of learning the relationship between inputs and known outputs, the algorithm analyses the data to identify hidden patterns, structures or relationships that may not be immediately apparent (Mitchell, 1997). This approach is particularly useful when labelled data are unavailable, limited or expensive to obtain.

The two most common unsupervised learning tasks are clustering and dimensionality reduction. Clustering groups similar observations based on shared characteristics, while dimensionality reduction reduces the number of variables in a dataset while preserving as much important information as possible (Bishop, 2006). Common unsupervised learning algorithms include K-Means, Hierarchical Clustering, Density-Based Spatial Clustering of Applications with Noise (DBSCAN) and Principal Component Analysis (PCA) (Goodfellow, Bengio and Courville, 2016). These algorithms have been widely applied in areas such as customer segmentation, anomaly detection, image processing, market analysis and bioinformatics, where discovering hidden patterns is often more important than predicting known outcomes.

Although unsupervised learning provides valuable insights into the underlying structure of data, its performance is often more difficult to evaluate than supervised learning because there are no labelled outputs against which the results can be compared. Consequently, interpreting the identified patterns often requires domain knowledge to determine whether they are meaningful and relevant to the problem being investigated (Bishop, 2006). In turbofan Remaining Useful Life (RUL) prediction, unsupervised learning can support tasks such as feature extraction, anomaly detection and dimensionality reduction. However, because it does not learn from known RUL values, it cannot directly predict Remaining Useful Life and is therefore typically used to support, rather than replace, supervised learning models.

2.3.1.3 Semi-supervised Learning

Semi-supervised learning is a machine learning paradigm that combines a small amount of labelled data with a larger amount of unlabelled data during the training process (Chapelle, Schölkopf and Zien, 2006). By learning from both types of data, the algorithm aims to improve predictive performance while reducing the need for extensive manual data labelling. This approach provides a balance between supervised and unsupervised learning, making it particularly suitable when labelled data are limited but unlabelled data are readily available.

Semi-supervised learning has been applied to both classification and regression problems across a range of domains, including medical diagnosis, speech recognition, image classification and natural language processing (Zhu and Goldberg, 2009). Common approaches include self-training, co-training, label propagation and graph-based methods (Chapelle, Schölkopf and Zien, 2006). These methods enable models to learn from a much larger pool of data than would be possible using labelled data alone, often improving predictive performance.

Despite its advantages, semi-supervised learning assumes that the labelled and unlabelled data follow similar underlying distributions. If this assumption is not satisfied, the additional unlabelled data may reduce rather than improve model performance. Consequently, the quality and representativeness of both labelled and unlabelled datasets are important when applying semi-supervised learning (Zhu and Goldberg, 2009). For turbofan Remaining Useful Life (RUL) prediction, semi-supervised learning is most useful when labelled degradation data are limited. However, because benchmark datasets such as NASA C-MAPSS provide labelled RUL values for model development, supervised learning is generally preferred and is the approach adopted in this dissertation.

2.3.1.4 Reinforcement Learning

Reinforcement learning is a machine learning paradigm in which an agent learns by interacting with an environment and receiving feedback in the form of rewards or penalties based on the actions it takes (Sutton and Barto, 2018). The agent uses this feedback to learn a strategy, known as

a policy, that maximizes cumulative rewards over time. Unlike supervised learning, reinforcement learning does not rely on labelled input-output pairs.

Reinforcement learning is primarily used for sequential decision-making problems where current actions influence future outcomes. Common reinforcement learning algorithms include Q-Learning, Deep Q-Networks (DQN), SARSA and Policy Gradient methods (Sutton and Barto, 2018). These algorithms have been successfully applied in robotics, autonomous vehicles, game playing, resource allocation and control systems, where adaptive decision-making is required under changing conditions.

Although reinforcement learning has achieved considerable success in complex decision-making tasks, it often requires extensive training, large amounts of interaction data and significant computational resources before achieving satisfactory performance (Goodfellow, Bengio and Courville, 2016). Because its objective is to learn optimal decisions rather than predict continuous target values, reinforcement learning is generally more suitable for optimization and control problems than for Remaining Useful Life (RUL) prediction. While it has potential applications in maintenance scheduling and control optimization, direct RUL estimation is fundamentally a supervised regression problem based on historical sensor data and known RUL values. Consequently, reinforcement learning falls outside the scope of this dissertation.

2.4 Machine Learning for RUL Prediction

Machine learning has become one of the most widely adopted approaches for Remaining Useful Life (RUL) prediction because of its ability to learn complex degradation patterns directly from historical sensor data without requiring explicit physical models (Lei et al., 2018).

In RUL prediction, the objective of machine learning is to estimate how much useful operating life remains before an engine reaches a predefined failure threshold. Unlike traditional physics-based and statistical methods, which often rely on predefined assumptions about equipment degradation, machine learning algorithms learn directly from historical observations, enabling them to model nonlinear relationships and adapt to different operating conditions (Si et al., 2011).

This capability makes machine learning particularly suitable for complex engineering systems such as turbofan engines, where degradation is influenced by multiple interacting factors and cannot easily be represented using fixed mathematical equations.

Machine learning models are trained using historical sensor measurements together with their corresponding Remaining Useful Life values to learn the relationship between engine condition and the remaining operating life. Because the objective is to predict a continuous numerical value, RUL prediction is generally formulated as a supervised regression problem (Lei et al., 2018).

As the availability of condition-monitoring and sensor data has increased, researchers have proposed a wide range of machine learning models for RUL prediction. However, studies have shown that prediction accuracy depends not only on the choice of learning algorithm but also on data quality, preprocessing techniques, feature engineering and model optimization (Zheng et al., 2017; Lei et al., 2018). Consequently, selecting an appropriate machine learning model remains an important area of research.

Based on the learning techniques employed, machine learning models used for RUL prediction are generally grouped into classical machine learning and deep learning approaches.

Classical machine learning models typically rely on manually engineered features extracted from sensor data before model training, whereas deep learning models automatically learn feature representations directly from sequential sensor data (LeCun, Bengio and Hinton, 2015).

Each approach has distinct strengths and limitations, and both have been widely investigated for turbofan Remaining Useful Life prediction.

The following sections critically review these two categories of machine learning models and evaluate their application to turbofan Remaining Useful Life prediction.

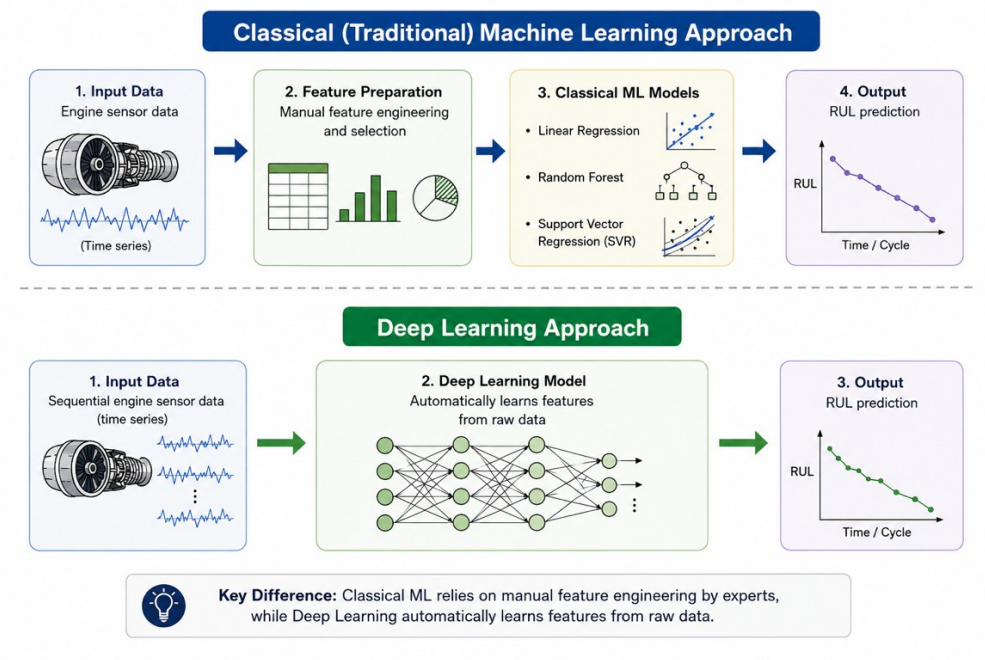


Figure Literature Review.6: Comparison of classical machine learning and deep learning approaches for RUL prediction. Source: Author's illustration based on LeCun, Bengio and Hinton (2015) and Goodfellow, Bengio and Courville (2016).

2.4.1 Classical Machine Learning Models

Classical machine learning models are algorithms that learn relationships between input features and target variables using statistical and mathematical techniques. Unlike deep learning models, they require manually engineered features extracted from the data before model training (LeCun, Bengio and Hinton, 2015).

Classical machine learning models have been widely applied to Remaining Useful Life (RUL) prediction because of their ability to learn relationships between historical sensor data and equipment degradation while requiring relatively low computational resources (Lei et al., 2018). These models formed the foundation of early data-driven RUL prediction research and continue to be used because they are generally easier to implement, interpret and evaluate than more complex deep learning approaches (Goodfellow, Bengio and Courville, 2016). As a result, many studies have adopted classical machine learning models as benchmark algorithms for assessing predictive performance.

Unlike deep learning models, classical machine learning algorithms typically rely on manually engineered features extracted from sensor measurements before model training (LeCun, Bengio and Hinton, 2015). Consequently, their predictive performance is influenced not only by the learning algorithm itself but also by the quality of data preprocessing, feature selection and feature engineering (Lei et al., 2018). While these models have demonstrated good predictive capability for RUL estimation, they generally have a limited ability to learn complex temporal dependencies directly from multivariate time-series data, making them less effective than deep learning models in some applications (Zheng et al., 2017).

Several classical machine learning algorithms have been investigated for turbofan RUL prediction, including Linear Regression, Support Vector Regression (SVR), Random Forest and Extreme Gradient Boosting (XGBoost). Each algorithm differs in its learning mechanism, predictive capability and computational requirements, resulting in varying levels of performance across different studies. The following sections critically review the principles, applications, strengths and limitations of the classical machine learning algorithms most used for turbofan Remaining Useful Life prediction.

2.4.1.1 Linear Regression

Linear Regression is a supervised machine learning algorithm used to predict continuous numerical values by modelling the relationship between one or more independent variables and a dependent variable (James et al., 2021).

The model assumes that the relationship between the input variables and the target variable can be approximated by a straight-line equation. It works by fitting a linear equation to the training data and using this relationship to predict unseen observations.

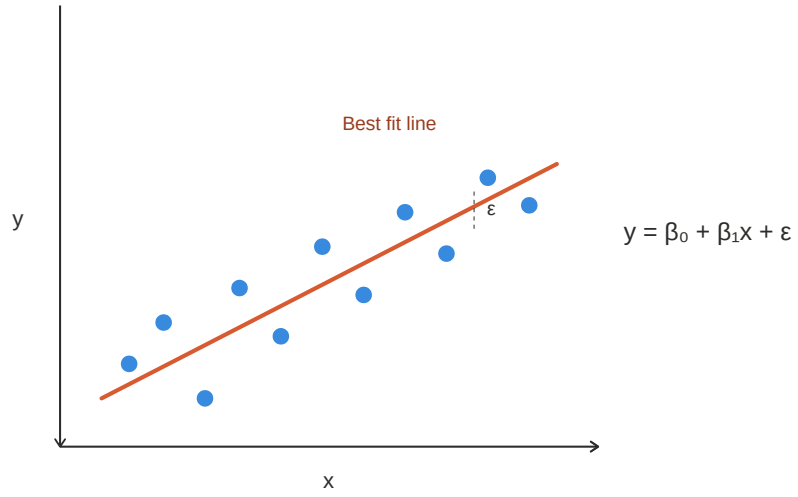


Figure Literature Review.7: Illustration of a linear regression best-fit line, showing the relationship between predictor (x) and target (y) variables (author's illustration).

Due to its simplicity and interpretability, Linear Regression has been widely adopted as a baseline model in Remaining Useful Life (RUL) prediction studies (Lei et al., 2018). However, because turbofan engine degradation is often nonlinear, the model struggles to accurately capture complex degradation patterns, resulting in lower predictive performance than more advanced machine learning algorithms (Zheng et al., 2017). Despite this limitation, Linear Regression remains valuable for benchmarking the performance of more sophisticated RUL prediction models.

2.4.1.2 Support Vector Regression (SVR)

Support Vector Regression (SVR) is a supervised machine learning algorithm that predicts continuous numerical values by identifying an optimal regression function within a specified error margin (Smola and Schölkopf, 2004).

Unlike Linear Regression, SVR can model both linear and nonlinear relationships using kernel functions that transform the data into a higher-dimensional space where nonlinear relationships can be modelled more effectively, making it more suitable for complex datasets.

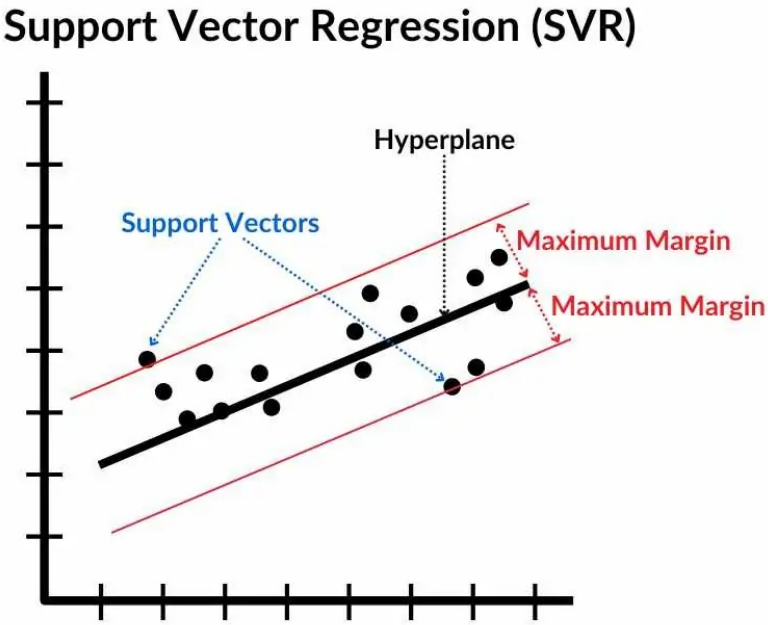


Figure Literature Review.8: Support vector regression illustrating the hyperplane, maximum margin, and support vectors (Van Otten, 2024).

Several studies have applied SVR to Remaining Useful Life (RUL) prediction and reported competitive predictive performance, particularly when combined with effective feature engineering and parameter tuning (Khelif et al., 2017; Lei et al., 2018).

However, the algorithm is sensitive to the choice of kernel and hyperparameters, and its computational cost increases with larger datasets. Despite these limitations, SVR remains one of the most widely used classical machine learning models for RUL prediction because of its ability to model nonlinear degradation patterns.

2.4.1.3 Random Forest

Random Forest is a supervised ensemble machine learning algorithm that combines multiple decision trees to improve prediction accuracy and reduce overfitting (Breiman, 2001).

During training, the algorithm generates multiple decision trees using different random subsets of the training data and input features. This approach enables Random Forest to capture complex and

nonlinear relationships within data more effectively than many traditional machine learning algorithms.

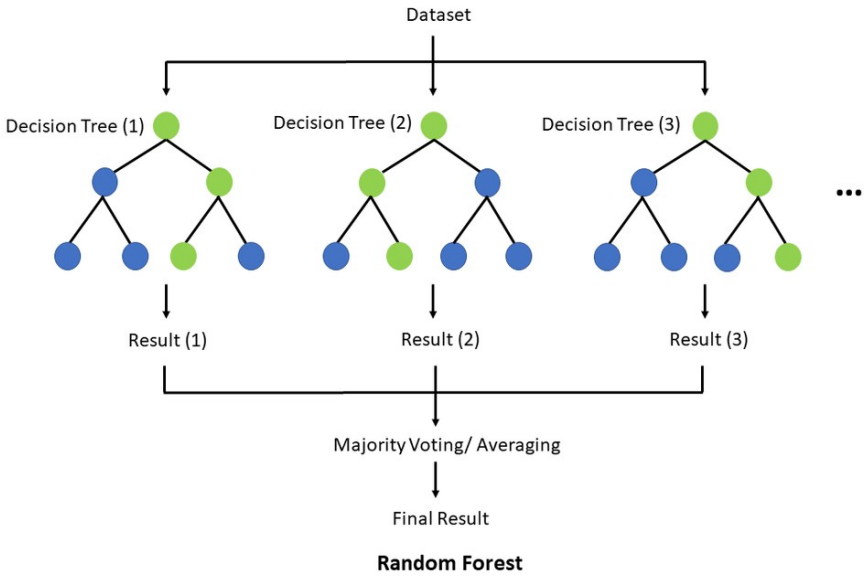


Figure Literature Review.9: Illustration of the random forest algorithm (Jagannath, 2017).

Previous studies have reported that Random Forest achieves strong predictive performance for Remaining Useful Life (RUL) prediction because of its robustness to noisy sensor data and ability to model complex degradation patterns (Wang et al., 2023; Lei et al., 2018). However, despite its high accuracy, the model relies on feature engineering and does not explicitly capture temporal dependencies in sequential sensor data.

Nevertheless, Random Forest remains one of the most widely adopted classical machine learning algorithms for RUL prediction due to its accuracy, robustness and ease of implementation.

2.4.1.4 Extreme Gradient Boosting (XGBoost)

Extreme Gradient Boosting (XGBoost) is a supervised ensemble machine learning algorithm that builds a series of decision trees sequentially with each new tree focusing on correcting the prediction errors made by the previous trees, gradually improving overall model performance. (Chen and Guestrin, 2016).

This boosting approach enables the model to achieve high predictive accuracy while efficiently handling complex and nonlinear relationships within data. XGBoost has been widely applied to Remaining Useful Life (RUL) prediction, with studies reporting strong performance due to its robustness, regularization techniques and ability to process large datasets efficiently (Jia et al., 2021).

However, the model requires careful hyperparameter tuning and, like other classical machine learning algorithms, relies on engineered features rather than learning directly from sequential sensor data. Despite these limitations, XGBoost is considered one of the most effective classical machine learning models for RUL prediction.

2.4.2 Non classical Models (Deep learning)

Non classical (deep learning) models is a subset of machine learning that uses artificial neural networks with multiple hidden layers to learn hierarchical representations directly from data (LeCun, Bengio and Hinton, 2015). Unlike classical machine learning, which relies on manually engineered features, deep learning automatically learns feature representations during training, enabling it to model complex nonlinear relationships and temporal dependencies directly from multivariate sensor data. These capabilities have made deep learning increasingly popular for Remaining Useful Life (RUL) prediction, particularly for turbofan engines, where degradation develops gradually over many operating cycles (Lei et al., 2018).

Consequently, many recent studies have reported improved predictive performance using deep learning models for RUL prediction (Zheng et al., 2017). Several deep learning architectures have been investigated for RUL prediction, with Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks and, more recently, Transformer models receiving considerable attention in the literature.

Although these models often achieve higher predictive accuracy than classical machine learning algorithms, they generally require larger datasets, greater computational resources and longer training times (LeCun, Bengio and Hinton, 2015). The following sections review these representative deep learning models and discuss their strengths and limitations for turbofan RUL prediction.

2.4.2.1 Recurrent Neural Networks (RNN)

Recurrent Neural Networks (RNNs) are deep learning models designed to process sequential data by retaining information from previous time steps (LeCun, Bengio and Hinton, 2015). This enables them to model temporal relationships more effectively than traditional machine learning algorithms. Early studies applied RNNs to Remaining Useful Life (RUL) prediction because of their ability to learn degradation trends from sequential sensor data.

However, conventional RNNs often suffer from the vanishing gradient problem, making it difficult to learn long-term dependencies (Goodfellow, Bengio and Courville, 2016). These limitations led to the development of more advanced recurrent architectures, particularly Long Short-Term Memory (LSTM) networks.

2.4.2.2 Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) is an advanced form of Recurrent Neural Network developed to overcome the vanishing gradient problem associated with traditional RNNs (Hochreiter and Schmidhuber, 1997).

LSTM achieves this through memory cells and gating mechanisms that regulate the flow of information, enabling the model to retain important information over long sequences while discarding irrelevant data. This makes LSTM particularly effective for modelling multivariate time-series data where historical observations influence future outcomes.

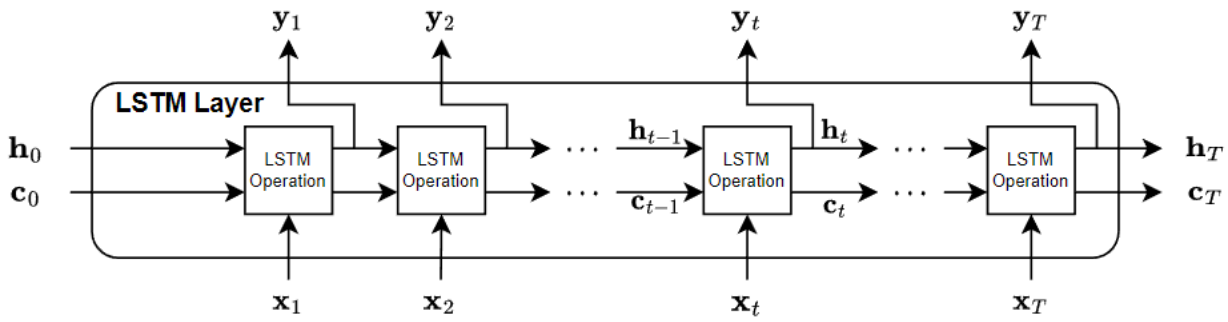


Figure Literature Review.10: Architecture of a Long Short-Term Memory (LSTM) unit showing the memory cell, input gate, forget gate and output gate. Source: MathWorks (n.d).

Because turbofan engine degradation occurs gradually over many operating cycles, LSTM has become one of the most widely adopted deep learning models for Remaining Useful Life (RUL) prediction. Zheng et al. (2017) reported that LSTM outperformed several traditional machine learning algorithms by effectively learning long-term degradation patterns directly from sensor data.

Similar findings have been reported in subsequent studies, which attribute the model's strong predictive performance to its ability to capture temporal dependencies without relying heavily on manual feature engineering (Lei et al., 2018). However, LSTM models generally require larger training datasets, longer training times and greater computational resources than classical machine learning algorithms. Despite these limitations, LSTM remains one of the most widely used deep learning models for turbofan RUL prediction and is therefore selected as the representative deep learning model in this study.

2.4.2.3 Transformer

Transformer models use attention mechanisms to capture relationships between observations regardless of their position in a sequence, enabling parallel processing during training (Vaswani et al., 2017). More recent studies have explored Transformers for RUL prediction because of their ability to model long-range dependencies in sensor data. Although they have shown promising results, Transformer models generally require larger datasets and greater computational resources than recurrent neural networks. As a result, their application to turbofan RUL prediction remains an active area of research.

2.5 Comparative Review of Machine Learning Studies for Turbofan RUL Prediction

Numerous studies have applied classical machine learning and deep learning models to turbofan Remaining Useful Life (RUL) prediction. However, comparing their reported performance is challenging because studies often use different datasets, preprocessing methods, evaluation metrics and experimental settings. Table 2.2 summarizes representative studies and highlights the models, datasets, evaluation metrics, key findings and limitations reported in the literature.

Table Literature Review.2:Summary of representative studies on machine learning models for turbofan Remaining Useful Life prediction, showing the datasets, models, evaluation metrics, key findings and limitations.

No.	Author(s) / Year	Dataset	Model / Approach	Metric(s)	Main result / finding	Limitation / relevance
1	Saxena and Goebel (2008)	NASA C-MAPSS	Benchmark dataset	RUL	Established C-MAPSS as a widely used simulated benchmark for RUL prediction.	Common benchmark, but simulated data limit real-world generalization.
2	Heimes (2008)	PHM08 turbofan challenge	RNN	PHM Score	Showed that RNNs could learn degradation patterns from sequential engine data.	Early RNNs were weaker at modelling long-term dependencies.
3	Peel (2008)	PHM08 turbofan challenge	Neural-network ensemble with Kalman filtering	PHM Score	Combined neural-network predictions with Kalman filtering for RUL estimation.	Effective but more complex than direct regression.
4	Ramasso and Saxena (2014)	C-MAPSS	Prognostic benchmarking	Multiple	Showed that results vary with operating conditions and evaluation methodology.	Supports the need for a controlled comparison framework.
5	Babu, Zhao and Li (2016)	C-MAPSS	Deep CNN	RMSE, NASA Score	Demonstrated automatic degradation-feature learning using deep CNNs.	Strong feature learning but limited explicit long-term temporal modelling.
6	Zheng et al. (2017)	C-MAPSS	LSTM	RMSE, NASA Score	Demonstrated effective temporal RUL modelling using LSTM.	Greater computational complexity and sensitivity to architecture choices.
7	Khelif et al. (2017)	Turbofan/C-MAPSS	SVR with variable selection	Prediction error	Showed competitive nonlinear RUL prediction using SVR.	Sensitive to feature selection and hyperparameter tuning.
8	Li, Ding and Sun (2018)	C-MAPSS	Deep CNN	RMSE, NASA Score	Demonstrated strong RUL prediction using sensor windows and deep CNNs.	Performance depends on preprocessing, window size and network design.

No.	Author(s) / Year	Dataset	Model / Approach	Metric(s)	Main result / finding	Limitation / relevance
9	Lei et al. (2018)	Multiple PHM datasets	Review of prognostic methods	Multiple	Concluded that RUL performance depends on the complete prognostic pipeline.	Supports evaluating preprocessing and modelling together.
10	Ellefsen et al. (2019)	C-MAPSS	Semi-supervised deep architecture with GA tuning	RMSE, NASA Score	FD001: RMSE 12.56; NASA Score 231.	Strong performance but greater training and optimization complexity.
11	Hou et al. (2020)	C-MAPSS	Autoencoder-based deep architecture	RMSE, NASA Score	FD001: RMSE 10.71; NASA Score 174.	High performance achieved using a complex multi-stage architecture.
12	Jia et al. (2021)	C-MAPSS	Decision Tree, RF, SVR and XGBoost	RMSE, MAE	XGBoost performed strongly against other classical models.	Relevant alternative, but outside this study's selected model set.
13	Wang, Cheng and Song (2021)	C-MAPSS	Transformer + TCNN	RMSE, NASA Score	FD001: RMSE 12.31; Score 252; ensemble RMSE 11.40.	Strong prediction but greater architectural complexity.
14	Asif et al. (2022)	C-MAPSS	LSTM with preprocessing and RUL labelling	RMSE, NASA Score	Best FD001 configuration: RMSE 7.78; Score 100.	Result depends strongly on preprocessing and model configuration.
15	Ensarioğlu, İnkaya and Emel (2023)	FD001	1D-CNN-LSTM with change-point labelling	RMSE, NASA Score	FD001: RMSE 16.1 ± 0.5; Score 437.2 ± 126.4.	Shows that RUL labelling and feature engineering affect reported performance.
16	Wang et al. (2023)	C-MAPSS FD001	RF feature selection + Bayesian-optimised MLP	RMSE and regression metrics	Reported a 6.1% reduction in FD001 RMSE.	RF was used for feature selection, not as the final predictor.
17	Liu et al. (2024)	C-MAPSS	Multiscale CNN + Transformer-based transfer learning	RMSE	FD001: RMSE 10.26; mean RMSE 12.45 across FD001–FD004.	Strong performance but substantially greater model complexity.

No.	Author(s) / Year	Dataset	Model / Approach	Metric(s)	Main result / finding	Limitation / relevance
18	Tippareddy (2024)	C-MAPSS and N-CMAPSS	BiLSTM	RMSE	FD001: RMSE 17.60.	Master's-project evidence rather than a peer-reviewed journal study.
19	Dida, Cheriet and Belhadj (2025)	C-MAPSS FD001/FD003	Attention-LSTM	RMSE, NASA Score	FD001: RMSE 12.33; NASA Score 200.	Attention adds model complexity and reduces transparency.
20	Elsherif et al. (2025)	C-MAPSS FD001/FD003	Convolutional autoencoder + Attention-LSTM	MAE, RMSE, NASA Score	FD001: MAE 10.49; RMSE 14.44; Score 282.38.	Strong performance but more complex than a standalone LSTM.

Although Table 2.2 includes quantitative results where these were clearly reported, the values should not be interpreted as a direct ranking of the models. The studies differ in C-MAPSS subset, RUL target construction, preprocessing, feature selection, sequence length, model configuration, hyperparameter optimization and evaluation procedure.

Consequently, a lower published RMSE or NASA score does not necessarily show that one algorithm is inherently superior. Instead, the table highlights broader patterns in predictive performance and methodological trade-offs.

These differences support the controlled comparison adopted in this dissertation, where the selected models are evaluated using a common dataset, preprocessing framework and final test set.

2.6 Challenges and Limitations

Machine learning has advanced Remaining Useful Life (RUL) prediction, yet several challenges continue to affect the reliability and practical application of existing models. A major concern is the widespread reliance on the NASA C-MAPSS dataset. Although it has become the standard benchmark for evaluating RUL prediction models, it is based on simulated engine degradation and may not fully represent the complexity of real operating environments, maintenance activities or

unexpected fault conditions (Saxena and Goebel, 2008). Consequently, findings obtained using C-MAPSS may not always generalize to operational aircraft engines.

Differences in experimental methodology further complicate comparisons between published studies. Variations in preprocessing, feature engineering, data splitting strategies and hyperparameter tuning mean that models are often evaluated under different conditions. As a result, it is difficult to determine whether improvements in predictive performance are due to the learning algorithm or the experimental methodology employed (Lei et al., 2018).

Another important challenge is the trade-off between predictive performance and practical implementation. Classical machine learning models are generally easier to interpret and require fewer computational resources, whereas deep learning models can learn complex temporal patterns directly from sequential sensor data but typically require larger datasets, longer training times and greater computational power. Consequently, selecting an appropriate model depends not only on predictive accuracy but also on the availability of data and computational resources.

2.7 Chapter Summary

This chapter reviewed the literature on machine learning for turbofan Remaining Useful Life (RUL) prediction. It introduced turbofan engines, predictive maintenance and the concept of RUL before examining machine learning, its learning paradigms, and the application of classical and deep learning models to RUL prediction.

The review showed that classical machine learning models, including Linear Regression, Support Vector Regression and Random Forest, remain widely used because of their simplicity, interpretability and computational efficiency, while deep learning models, particularly Long Short-Term Memory (LSTM) networks, have demonstrated strong predictive performance by learning temporal degradation patterns from sequential sensor data.

The literature also highlighted that no single model consistently outperforms all others, as predictive performance is influenced by factors such as dataset characteristics, preprocessing methods and evaluation strategies. This review identified the need for fair and consistent model comparison under a common experimental framework. The next chapter presents the methodology

adopted to address this need, including the research design, dataset, data preprocessing, model development and evaluation procedures.

3 Methodology

3.1 Introduction

This chapter describes the methodology adopted to compare five prediction models for turbofan Remaining Useful Life (RUL) prediction. It outlines the research design, dataset selection, data-preparation procedures, model development, validation, and evaluation methods used throughout the study.

A controlled experimental framework was adopted to support a fair comparison of the selected models, while allowing model-specific training and validation procedures where required by the characteristics of each modelling approach.

The chapter explains and justifies these methodological choices, while the subsequent chapters present their implementation and results.

3.2 Overall Research Workflow

Figure 3.1 summarizes the overall workflow adopted in this study. It shows the sequence from research design and data preparation through model development, validation and final evaluation. The same overall data pipeline was followed across the models, although some training and validation stages differed according to the requirements of each model.

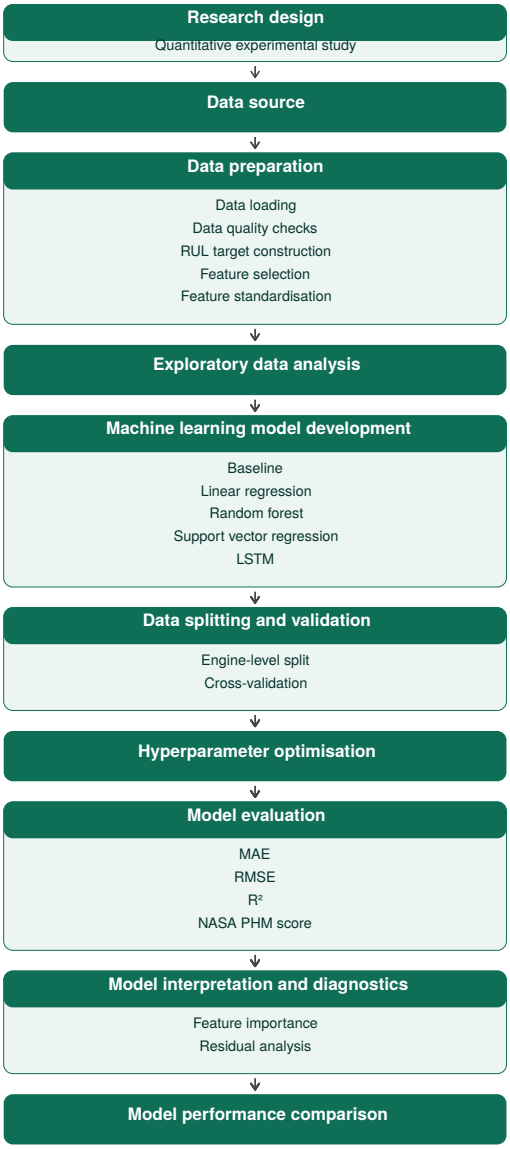


Figure Methodology.11: Overall research workflow.

The workflow provided a structured process for developing and evaluating the models while maintaining separation between development data and the official NASA test set.

3.3 Research Design

The study was comparative because it evaluated a baseline approach and four established machine learning models rather than proposing a new algorithm. The models used the same FD001 dataset, RUL target definition, retained predictors, engine-level development split, official NASA test set and final evaluation metrics.

However, their development procedures were not identical. The classical models were assessed using grouped cross-validation, with Random Forest and SVR also undergoing grouped hyperparameter optimization, whereas the LSTM used held-out engine validation and early stopping. This approach maintained consistent final testing conditions while allowing each model to be developed using procedures suited to its characteristics.

3.4 Data Source and Dataset Description

The NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) dataset is a widely used benchmark for Remaining Useful Life (RUL) prediction using simulated turbofan engine degradation data (Saxena et al., 2008).

The dataset contains four subsets that differ in operating conditions and fault modes; this study used FD001, summarized in Table 3.1. Each record represents the condition of a single engine at one operating cycle. Training engines were run to failure, whereas test engines were truncated before failure and their true RUL values were provided separately.

FD001 was selected because it contains a single operating condition and a single fault mode, reducing variability introduced by multiple operating conditions and fault modes (Saxena et al., 2008; Ramasso and Saxena, 2014). This allowed differences in predictive performance to be attributed more confidently to the learning algorithms rather than the complexity of the dataset. However, this simplicity also limits the generalizability of the findings to the more complex operating conditions represented by FD002–FD004 (Ramasso and Saxena, 2014).

Table Methodology.3: Summary of the C-MAPSS FD001 dataset used in this study.

Source	NASA C-MAPSS	Standard benchmark dataset for RUL prediction
Subset	FD001	Selected for this study
Training engines	100	Complete run-to-failure trajectories
Test engines	100	Used for final model evaluation
Training records	20,631	Sufficient observations for model training
Operating conditions	1	Removes variability between operating regimes
Fault modes	1 (HPC degradation)	Simplifies comparison across models
Features per record	26 (Engine ID, Cycle, 3 settings, 21 sensors)	Multivariate sensor data for prediction
Target variable	Remaining Useful Life (RUL)	Continuous regression target
Data type	Multivariate time-series	Suitable for both classical ML and LSTM

3.5 Data Processing

3.5.1 Data Quality Checks

The training and test datasets were subjected to data quality checks before preprocessing and model development. The datasets were examined for missing values, duplicate records, unexpected dimensions, column consistency, and the expected number of engines and sensor variables.

These checks were performed to verify the integrity of the data and to reduce the risk of errors that could affect model training and evaluation. Numerical summaries and a missingness map were used to assess data completeness across all variables.

3.5.2 RUL Target Construction

The Remaining Useful Life (RUL) target variable was constructed because the NASA C-MAPSS training dataset does not provide RUL values directly (Saxena et al., 2008). RUL was calculated as the difference between an engine's final cycle and its current cycle.

However, this approach assumes that RUL decreases from the first recorded cycle, even though measurable degradation may not be evident during the early stages of engine operation (Heimes, 2008).

To reduce the influence of this assumption, RUL values were capped at 125 cycles following Heimes (2008), allowing the models to focus on the later stages of engine life where degradation is more clearly reflected in the sensor measurements. The capped RUL values were calculated using Equation (3.1).

$$RUL_i = min(t_{max} - t_i, 125) \quad (3.1)$$

3.5.3 Feature Scaling (Standardization)

Feature standardization was applied to ensure that differences in numerical range did not disproportionately influence models that are sensitive to feature magnitude. The `StandardScaler` was fitted using only the training data and subsequently applied to the validation and test datasets using the same scaling parameters to prevent data leakage (Kaufman, Rosset and Perlich, 2012).

Standardization was required for SVR and LSTM because their training is sensitive to feature magnitude. The same transformation was retained within the Linear Regression pipeline for consistency and numerical comparability, although ordinary least-squares prediction is not inherently dependent on feature scale.

Random Forest was trained using the original feature values, as tree-based models are generally unaffected by differences in feature scale. A separate scaler was used for exploratory data analysis and was not reused during model development or evaluation.

$$x' = (x - \mu) / \sigma \quad (3.2)$$

3.5.4 Feature Selection

Feature selection was performed to identify predictors that were unlikely to contribute useful information to the models. Variables with little or no variation were considered for removal because they provide limited discriminatory information and are unlikely to improve predictive

performance. As the sensor measurements and operational settings differ in their numerical characteristics, separate criteria were used to assess each group.

Sensor variables with a standard deviation below 0.01 were considered near constant and removed, while operational settings were evaluated using the number of distinct values, with settings containing only a single distinct value excluded. Engine ID was also removed because it is an identifier rather than a predictive variable. The remaining variables were retained for model development.

3.6 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was conducted to understand the characteristics of the FD001 dataset before model development. EDA is commonly used to summarize datasets, examine variable distributions, identify potential anomalies and support data preprocessing prior to machine learning model development (Han, Kamber and Pei, 2012; Kuhn and Johnson, 2013).

To ensure that variables with different numerical ranges could be compared consistently during exploratory analysis, the retained predictors were standardized to zero mean and unit variance using Equation (3.2).

This standardization was performed solely for exploratory analysis and was not reused during model development, where a separate leakage-safe scaling procedure was applied as described in Section 3.5.3.

3.6.1 Distribution of the Target Variable

The distribution of the Remaining Useful Life (RUL) target variable was examined using descriptive statistics, a histogram and a boxplot.

This analysis was performed to understand the central tendency, variability and overall distribution of the target variable before model development, as well as to identify potential outliers that could influence model performance (Field, 2018; James et al., 2021).

3.6.2 Normality Assessment

The normality of the RUL distribution was assessed using a Quantile–Quantile (QQ) plot together with the Shapiro–Wilk and Kolmogorov–Smirnov tests. This assessment was performed to

determine how closely the RUL values followed a normal distribution and to provide a descriptive understanding of the dataset before model development (Field, 2018).

The QQ plot provided a graphical assessment of normality, while the Shapiro–Wilk and Kolmogorov–Smirnov tests provided statistical evidence of departures from normality. As formal normality tests are highly sensitive to large sample sizes, the graphical and statistical assessments were interpreted together (Ghasemi and Zahediasl, 2012).

However, because Linear Regression assumes normally distributed residuals rather than a normally distributed target variable, the normality assessment was used to describe the dataset rather than determine model selection (Kuhn and Johnson, 2013).

3.6.3 Sensor Trend Analysis

Sensor trend plots were generated to examine how selected sensor measurements changed over successive engine cycles. This analysis was performed to identify degradation patterns and understand how sensor behaviour evolved throughout an engine's operational life before model development.

As the NASA C-MAPSS dataset consists of multivariate time-series sensor measurements collected until engine failure, visualizing these trends provided insight into how the sensor readings changed as engines degraded (Saxena et al., 2008).

3.6.4 Correlation Analysis

Pearson correlation analysis was performed to examine the linear relationship between each predictor variable and the Remaining Useful Life (RUL) target.

The correlation coefficients were visualized using a correlation heatmap to identify the strength and direction of the relationships between the predictor variables and RUL (James et al., 2021).

This analysis was conducted to provide an initial understanding of the variables most closely associated with engine degradation and to identify predictors that might contain overlapping information.

However, the results were used for exploratory purposes only and were not treated as the sole basis for feature selection, as weak linear correlations do not necessarily indicate low predictive value and correlation does not imply causation (Kuhn and Johnson, 2013; Field, 2018).

3.7 Machine Learning Models

Five models were selected to predict the Remaining Useful Life (RUL) of turbofan engines: a baseline model, Linear Regression, Random Forest, Support Vector Regression (SVR) and a Long Short-Term Memory (LSTM) network.

These models were chosen because they represent a range of classical machine learning and deep learning approaches that have been widely applied to RUL prediction in previous studies.

Comparing the models under the same experimental conditions enabled their predictive performance to be evaluated consistently using a common dataset, preprocessing pipeline and evaluation protocol. The following sections outline the rationale for selecting each model.

3.7.1 Mean RUL Baseline

The Mean RUL Baseline was included as a reference model to provide a minimum performance benchmark for comparison with the machine learning models.

The baseline predicts the mean Remaining Useful Life (RUL) of the training data for every observation without using the sensor measurements.

Comparing the machine learning models against this simple approach allowed the predictive value of the more advanced algorithms to be evaluated, as a useful model should achieve lower prediction error than the baseline (James et al., 2021).

3.7.2 Linear Regression

Linear Regression was selected as a benchmark model because it is computationally efficient, easy to interpret and widely used in regression problems (James et al., 2021).

The model estimates the relationship between the predictor variables and the Remaining Useful Life (RUL) target by fitting a linear function to the training data. Including Linear Regression

provided a simple reference against which the performance of more advanced machine learning models could be compared.

However, because turbofan engine degradation is often characterized by complex non-linear relationships, the predictive performance of Linear Regression was expected to be limited.

3.7.3 Random Forest

Random Forest was selected because it can model complex non-linear relationships, capture interactions between predictor variables and is robust to noisy data (Breiman, 2001).

The model generates multiple decision trees using random subsets of the training data and predictor variables, with the final prediction obtained by averaging the predictions of all trees.

Unlike scale-sensitive algorithms, Random Forest does not require feature standardization, making it well suited to the sensor measurements used for Remaining Useful Life (RUL) prediction.

3.7.4 Support Vector Regression (SVR)

Support Vector Regression (SVR) was selected because it can model complex non-linear relationships between the predictor variables and the Remaining Useful Life (RUL) target, making it well suited to turbofan engine degradation data (Smola and Schölkopf, 2004).

The model estimates a regression function by allowing small prediction errors within a predefined margin and can capture non-linear patterns using kernel functions.

As SVR is sensitive to differences in feature magnitude, feature standardization was applied before model training. Its performance also depends on appropriate hyperparameter selection, which was addressed through hyperparameter optimization.

3.7.5 Long Short-Term Memory (LSTM)

A Long Short-Term Memory (LSTM) network was selected because it is designed to learn temporal patterns from sequential data, making it well suited to Remaining Useful Life (RUL) prediction using multivariate time-series sensor measurements (Hochreiter and Schmidhuber, 1997).

Unlike the classical machine learning models, the LSTM was trained using sequences of sensor observations, enabling it to learn how engine behaviour evolved over successive operating cycles.

Previous studies have demonstrated that LSTM networks are effective for modelling temporal degradation patterns and predicting RUL from the NASA C-MAPSS dataset (Zheng et al., 2017).

3.8 Data Splitting and Validation Strategy

The official NASA C-MAPSS FD001 training dataset was first split at the engine level, with 80 engines used for model development and 20 engines reserved for held-out validation. Keeping all operating cycles from the same engine within one partition prevented observations from the same engine appearing in both development and validation data, reducing the risk of data leakage (Kuhn and Johnson, 2013).

The 80-engine development partition was used for model fitting and hyperparameter optimization. For Random Forest and SVR, five-fold GroupKFold was used within this partition during RandomizedSearchCV, with engine ID used as the grouping variable. The 20 held-out engines were not used during this tuning process and were subsequently used to evaluate the developed models on unseen engine trajectories.

After the held-out evaluation, an additional five-fold GroupKFold assessment was performed across all 100 engines in the official FD001 training dataset for the Mean Baseline and classical machine learning models. This stage was used to examine whether their performance remained stable across different engine groupings rather than relying only on the original 80:20 split. The LSTM was not included in this grouped cross-validation and was instead assessed using the held-out validation set, training and validation loss, and early stopping.

The official NASA test dataset remained separate from all model development, tuning and validation procedures. After the final model configurations were selected, the models were retrained using all 100 FD001 training engines and evaluated once on the 100 official test engines. This test set therefore provided the independent final assessment of model performance.

3.9 Hyperparameter Optimization

Hyperparameter optimization was performed for the Random Forest and Support Vector Regression (SVR) models to identify parameter values that improved predictive performance. Unlike model parameters, which are learned during training, hyperparameters are specified before training and influence how a model learns from the data (James et al., 2021).

`RandomizedSearchCV` was selected because it evaluates a randomly selected subset of parameter combinations rather than every possible combination, reducing computational cost while providing an efficient search of the parameter space (Bergstra and Bengio, 2012).

Hyperparameter optimization was performed using 5-fold GroupKFold cross-validation, with the engine identifier used as the grouping variable to ensure that observations from the same engine remained within the same fold, thereby preventing data leakage during model tuning. The search aimed to minimize the Root Mean Squared Error (RMSE) using `scikit-learn's` negative RMSE scoring function.

The Random Forest search covered the number of trees, maximum tree depth, minimum samples per split and minimum samples per leaf, giving 108 possible combinations, of which ten were sampled.

The SVR search covered the penalty parameter C, the kernel width gamma and the tolerance band epsilon. An initial search over a narrower range returned C and gamma at the limits of the values offered, indicating that a stronger configuration might lie beyond the range tested; the space was therefore widened to include larger C values and smaller gamma values, giving 168 possible combinations, of which twenty were sampled. Under the widened space both parameters were selected from within the interior of the range rather than at its boundary.

The best-performing hyperparameter combination identified within the predefined search space was used to train the final Random Forest and SVR models. The LSTM model was trained using a fixed architecture with early stopping, and a separate hyperparameter search was not performed because of the additional computational cost. Early stopping monitored validation loss and restored the weights of the best-performing epoch, so that training halted once validation performance stopped improving rather than continuing until the epoch limit was reached.

3.10 Model Evaluation

Model evaluation was performed to assess how accurately each model predicted the Remaining Useful Life (RUL) of previously unseen engines.

Following model development and hyperparameter optimization, the final models were retrained using the complete NASA C-MAPSS FD001 training dataset and evaluated on the official NASA test dataset, which remained separate throughout model development (Saxena et al., 2008).

This provided an unbiased assessment of each model's ability to generalize to unseen engine trajectories.

Following the standard NASA C-MAPSS evaluation protocol, one final RUL prediction was generated for each test engine and compared with its corresponding ground-truth RUL value. For the classical models, this prediction was based on the final observed cycle, while the LSTM used the final 30-cycle sequence ending at that cycle.

Model performance was evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), the coefficient of determination (R^2) and the NASA PHM Score. These metrics were selected to assess prediction accuracy; explain the proportion of variance captured by each model and evaluate prediction errors using both symmetric and application-specific performance measures.

3.11 Performance Evaluation Metrics

Model performance was evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), the coefficient of determination (R^2), and the NASA PHM Score.

These metrics were selected because they assess different aspects of predictive performance, providing a more comprehensive evaluation than any single measure alone.

MAE and RMSE quantify prediction error, R^2 assesses the proportion of variation explained by the model, and the NASA PHM Score provides an application-specific measure by penalizing overestimation of engine life more heavily than underestimation (Saxena et al., 2008). The evaluation metrics used in this study are summarized in Table 3.2.

Table Methodology.4: Performance evaluation metrics used in this study.

Metric	What it measures	Why it was used
MAE	Average absolute prediction error.	Easily interpretable measure of average prediction error in the same units as RUL.
RMSE	Square root of the average squared prediction error.	Penalizes larger prediction errors more heavily than MAE.
R^2	Proportion of variation in RUL explained by the model.	Indicates the overall goodness-of-fit of the model.
NASA PHM Score	Asymmetric error metric for RUL prediction.	Reflects the greater operational risk of overestimating engine life.

3.12 Model Interpretation and Diagnostic Analysis

Model interpretation and diagnostic analysis were performed to better understand how the machine learning models generated predictions and to evaluate the quality of their results. These analyses were conducted to identify the variables that contributed most to the predictions, assess prediction errors and determine whether the models generalized well to unseen data.

For the Random Forest model, feature importance was calculated to identify the sensor measurements that contributed most to the model's predictions (Breiman, 2001). Pearson correlation analysis was also used to examine the linear relationship between each predictor and the Remaining Useful Life (RUL) target. Comparing feature importance with correlation coefficients provided additional insight into the relationship between predictor variables and model behaviour. However, these analyses were used for interpretation only and were not used to infer causal relationships.

Model diagnostics were performed using actual-versus-predicted plots, residual plots and residual distributions. These visualizations were examined to identify systematic prediction errors, assess potential bias and determine whether the prediction errors followed expected patterns. During

model development, training and validation performance, together with the LSTM learning curves, were monitored to identify signs of overfitting and evaluate how well the models generalized beyond the training data (James et al., 2021).

3.13 Chapter Summary

This chapter described the methodology used to compare five prediction models for turbofan Remaining Useful Life (RUL) prediction using the NASA C-MAPSS FD001 dataset.

It explained the research design, data preparation procedures, model development process, validation strategy and evaluation framework used to ensure a consistent and fair comparison across all models.

The chapter also outlined the hyperparameter optimization, performance evaluation metrics and diagnostic analyses used to assess and interpret model performance.

Using consistent data preparation and final evaluation conditions across the models, while allowing model-specific training and validation procedures where required, supported a fair comparison of their predictive performance.

The next chapter describes the implementation of this methodology in Python.

4 Implementation

4.1 Introduction

This chapter describes how the methodology presented in Chapter 3 was implemented in Python. It follows the same structure as the methodology so that each stage of the machine learning pipeline can directly be linked to its corresponding implementation.

Code screenshots included throughout this chapter were taken directly from the Python script, `rul_prediction.py`, which was executed to produce all outputs used in this study. This chapter focuses on describing the implementation of the machine learning pipeline, while the results produced by the implementation are presented and discussed in the subsequent chapter.

4.2 Overall Research Workflow

The workflow presented in Figure 3.1 was implemented as a single Python script, `rul_prediction.py`, which executes sequentially from the first stage to the last. The script is organized into clearly labelled steps that are printed to the console during execution, making it straightforward to identify the current stage of the pipeline.

At the end of each step, the relevant outputs, including tables, figures and trained models, are saved before the next stage begins. Figure 4.1 shows the overall structure of the implementation.

```
# STEP 1: DATA LOADING
# STEP 2: DATA PREPROCESSING
# STEP 3: EXPLORATORY DATA ANALYSIS
# STEP 4: ENGINE-LEVEL DEVELOPMENT SPLIT
# STEP 5: MODEL DEVELOPMENT
# STEP 6: HYPERPARAMETER OPTIMISATION
# STEP 7: VALIDATION AND CROSS-VALIDATION
# STEP 8: FINAL RETRAINING
# STEP 9: OFFICIAL NASA TEST EVALUATION
# STEP 10: MODEL INTERPRETATION
# STEP 11: SAVE OUTPUTS
```

Figure Implementation.12: The eleven main steps of rul_prediction.py, in the order they run.

4.3 Script Initialization

Before the machine learning pipeline begins, the script imports the required Python libraries used throughout the implementation, including pandas for data handling, NumPy for numerical operations, scikit-learn for machine learning, TensorFlow for the LSTM model and Matplotlib for visualization.

Global variables are then defined for the random seed, data directory and output directory, allowing these settings to be reused consistently throughout the script. To support reproducibility, the random seed is fixed at 42 for Python, NumPy and TensorFlow, deterministic TensorFlow operations are enabled, and the required output folders are created automatically if they do not already exist. Figure 4.2 shows the script initialization performed before data loading begins.

```
43
44  # --- Global settings (used across multiple steps) ----------------------------
45  SEED = 42         # Reproducibility seed for all random operations
46  DATA_DIR = "Data"        # Folder containing the NASA C-MAPSS CSV files
47  OUTPUT_DIR = "Outputs"   # Root folder for all generated outputs
48  os.environ["PYTHONHASHSEED"] = str(SEED)
49  os.environ["TF_DETERMINISTIC_OPS"] = "1"
50  random.seed(SEED)
51  np.random.seed(SEED)
52  tf.random.set_seed(SEED)
53  tf.config.experimental.enable_op_determinism()
```

Figure Implementation.13: Script initialization showing the global configuration, reproducibility settings and automatic creation of the output directories.

4.4 Data Loading

The three FD001 datasets (`train_FD001.csv`, `test_FD001.csv` and `RUL_FD001.csv`) were loaded into separate pandas Data Frames using the `read_csv()` function.

After loading, the script displayed the first and last records, dataset dimensions, column names and data types to verify that the files had been imported correctly before preprocessing began. Figure 4.3 shows the implementation of the data loading process.

```
train_data = pd.read_csv(os.path.join(DATA_DIR, "train_FD001.csv"),
                         encoding="utf-8-sig")

test_data = pd.read_csv(os.path.join(DATA_DIR, "test_FD001.csv"),
                        encoding="utf-8-sig")

rul_data = pd.read_csv(os.path.join(DATA_DIR, "RUL_FD001.csv"),
                       encoding="utf-8-sig")
```

Figure Implementation.14: Loading the three FD001 files and checking their structure.

4.5 Data Processing

4.5.1 Data Quality Checks

A missingness map was generated for both the training and test datasets using a reusable plotting function. The function checks each dataset for missing values, creates a heatmap and saves the figure automatically to the output directory. Figure 4.4 shows the implementation of the missingness map function.

```
def plot_missingness(df, title, filename):
    fig, ax = plt.subplots(figsize=(12, 5))
    sns.heatmap(df.isnull(), cbar=False, yticklabels=False,
                cmap=["#2c3e50", "#e74c3c"], ax=ax)
    ax.set_title(title)
    ax.set_xlabel("Variable")
    fig.savefig(os.path.join(OUTPUT_DIR, "Figures", filename),
                dpi=300, bbox_inches="tight")
    plt.close()
```

Figure Implementation.15: The function used to generate and save the missingness maps for the training and test data

4.5.2 RUL Target Construction

The script calculated the Remaining Useful Life (RUL) for each training record by first identifying the maximum operating cycle for every engine. This value was then merged with the training dataset and used to calculate the RUL by subtracting the current cycle from the maximum cycle. The calculated RUL values were capped at 125 cycles before the temporary `max_cycle` column was removed. Figure 4.5 shows the implementation of the RUL target construction.

```
print("\n--- RUL Target Construction ---")
max_cycles = train_data.groupby("engine_id")["cycle"].max()
train_data = train_data.merge(max_cycles.rename("max_cycle"),
                              on="engine_id", how="left")
train_data["RUL"] = train_data["max_cycle"] - train_data["cycle"]
train_data["RUL"] = train_data["RUL"].clip(upper=RUL_CAP)
train_data = train_data.drop(columns=["max_cycle"])
```

Figure Implementation.16: Implementation of the Remaining Useful Life (RUL) target construction

4.5.3 Feature Scaling (Standardization)

Standardization for Linear Regression and Support Vector Regression (SVR) was implemented using scikit-learn pipelines, with a `StandardScaler` applied before the model in each pipeline. This approach ensures that feature scaling is performed automatically whenever either model is fitted. Random Forest was implemented without a scaling step, while the LSTM uses a separate scaler during its own training process. Figure 4.6 shows the implementation of the Linear Regression and SVR pipelines.

```
lr_pipeline = Pipeline([("scaler", StandardScaler()),
                        ("model", LinearRegression())])

svr_pipeline = Pipeline([("scaler", StandardScaler()),
                         ("model", SVR(kernel="rbf"))])
```

Figure Implementation.17: The scaling pipelines used for Linear Regression and SVR.

4.5.4 Feature Selection

The script identified the sensor and operational setting variables before calculating the standard deviation of each sensor and the number of unique values for each operational setting. These values were used to identify constant predictors according to the predefined feature selection criteria. The identified constant sensors, constant settings and the engine identifier were then removed from the dataset before the remaining predictors were prepared for the subsequent stages of the pipeline. Figure 4.7 shows the implementation of this feature selection process.

```
157  sensor_cols = [col for col in train_data.columns if col.startswith("sensor_")]
158  setting_cols = [col for col in train_data.columns if col.startswith("setting_")]
159
160  sensor_std = train_data[sensor_cols].std().sort_values()
161  setting_nunique = train_data[setting_cols].nunique().sort_values()
162
163  # Save both variability tables as evidence for the removal decision
164  sensor_std.rename("Std_Dev").to_csv(
165      os.path.join(OUTPUT_DIR, "Tables", "sensor_variability.csv"))
166  setting_nunique.rename("Unique_Values").to_csv(
167      os.path.join(OUTPUT_DIR, "Tables", "setting_variability.csv"))
168  print("  Sensor and setting variability tables saved.")
169
170  STD_THRESHOLD = 0.01
171  constant_sensors = sensor_std[sensor_std < STD_THRESHOLD].index.tolist()
172  constant_settings = setting_nunique[setting_nunique == 1].index.tolist()
173  constant_predictors = constant_sensors + constant_settings
174
```

Figure Implementation.18: Implementation of the sensor and operational setting variability checks.

4.6 Exploratory Data Analysis

Before the exploratory analysis was performed, a standardized copy of the predictor data was created using z-score standardization with `StandardScaler` for analyses where a common scale was useful. Original-scale values were retained where direct interpretation of sensor measurements was required. Figure 4.8 shows the implementation of the standardization step.

```
231  print("\n--- Feature Normalization (for EDA) ---")
232  eda_scaler = StandardScaler().fit(X)
233  X_scaled = pd.DataFrame(eda_scaler.transform(X), columns=feature_cols, index=X.index)
234
```

Figure Implementation.19: Standardizing the predictors before any exploratory analysis is produced.

4.6.1 Distribution of the Target Variable

A histogram and a boxplot of the RUL target were generated from the processed training dataset. Both figures were saved automatically to the output directory for use in the subsequent analysis. Figure 4.9 shows the implementation of these visualizations.

```
# RUL histogram
fig, ax = plt.subplots(figsize=(7, 4))
sns.histplot(train_data["RUL"], bins=30, kde=True, ax=ax)
ax.set_title("Distribution of RUL (capped at 125)")
ax.set_xlabel("RUL")
fig.savefig(os.path.join(OUTPUT_DIR, "Figures", "rul_histogram.png"),
            dpi=300, bbox_inches="tight")
plt.close()
print("RUL histogram saved.")

# RUL boxplot
fig, ax = plt.subplots(figsize=(7, 3))
sns.boxplot(x=train_data["RUL"], ax=ax)
ax.set_title("Boxplot of RUL")
```

Figure Implementation.20: Generating the RUL histogram and boxplot.

4.6.2 Normality Assessment

The script performed two statistical normality tests on the RUL target. The Shapiro–Wilk test was applied to a random sample of 5,000 observations, while the Kolmogorov–Smirnov test was performed using the complete RUL dataset. The calculated test statistics and p-values were then saved for the subsequent analysis. Figure 4.10 shows the implementation of the statistical normality tests.

```
# Normality tests
shapiro_stat, shapiro_p = stats.shapiro(
    np.random.RandomState(SEED).choice(y.values, size=5000, replace=False))
ks_stat, ks_p = stats.kstest(y.values, "norm",
                             args=(y.mean(), y.std()))
```

Figure Implementation.21: Implementation of the statistical normality tests.

4.6.3 Sensor Trend Analysis

The script selected Engine 1 from the training dataset and plotted the values of sensor_2 against the operating cycle to illustrate the sensor trend over time. The completed figure was then saved automatically to the output directory for use in the subsequent analysis. Figure 4.11 shows the implementation of the sensor trend plot.

```
engine_1 = train_data[train_data["engine_id"] == 1]
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(engine_1["cycle"], engine_1["sensor_2"])
ax.set_title("Sensor 2 Trend Over Time – Engine 1")
ax.set_xlabel("Cycle")
ax.set_ylabel("Sensor 2")
fig.savefig(os.path.join(OUTPUT_DIR, "Figures", "sensor_trend_example.png"),
            dpi=300, bbox_inches="tight")
plt.close()
print("Sensor trend plot saved.")
```

Figure Implementation.22: Implementation of the sensor trend analysis

4.6.4 Correlation Analysis

Pearson correlation coefficients were calculated between the predictor variables and the RUL target using the processed training data. The resulting correlation matrix was visualised as a heatmap and saved automatically to the output directory for the subsequent analysis. Figure 4.12 shows the implementation of the correlation heatmap.

```
# Full correlation heatmap (computed on normalized predictors + RUL)
fig, ax = plt.subplots(figsize=(12, 9))
corr_matrix = pd.concat([X_scaled, y], axis=1).corr()
sns.heatmap(corr_matrix, cmap="coolwarm", ax=ax)
ax.set_title("Correlation Heatmap")
```

Figure Implementation.23: Implementation of the Pearson correlation heatmap.

The outputs generated during the exploratory analysis are presented and discussed in Chapter 5. The next stage of the implementation was the development of the machine learning models.

4.7 Machine Learning Models

This section describes the implementation of the five prediction models used in this study. Each model was defined using the same predictor variables and the same RUL target, providing a consistent implementation framework for subsequent training and evaluation. This section focuses on how each model was implemented, while the training process, validation and performance evaluation are presented in the following sections and Chapter 5.

4.7.1 Mean RUL Baseline

The Mean RUL Baseline was implemented by assigning the mean RUL of the training data as the prediction for every validation observation. This provided a simple reference model against which the machine learning models were later compared. Figure 4.13 shows the implementation of the baseline prediction. The performance of the baseline model is presented in Section 5.7.1.

```
488    baseline_pred = np.full(len(y_val), y_train.mean())
```

Figure Implementation.24: Implementation of the Mean RUL Baseline prediction.

4.7.2 Linear Regression

Linear Regression was implemented as a scikit-learn pipeline consisting of a `StandardScaler` followed by a `LinearRegression` model. This pipeline applies feature standardization automatically before fitting the model. Figure 4.14 shows the implementation of the Linear Regression pipeline. The performance of the model is presented in Section 5.7.2.

```
411    lr_pipeline = Pipeline([("scaler", StandardScaler()),
412                            ("model", LinearRegression())])
413
```

Figure Implementation.25: Implementation of the Linear Regression pipeline.

4.7.3 Random Forest

Random Forest was implemented using scikit-learn's `RandomForestRegressor` with a fixed random seed and parallel processing enabled. The model definition was created before the hyperparameter optimization stage described later in the implementation. Figure 4.15 shows the implementation of the Random Forest model. The performance of the model is presented in Section 5.7.3.

```
417    # n_jobs=-1 uses all available CPU cores for parallel tree training
418    rf_model = RandomForestRegressor(random_state=SEED, n_jobs=-1)
```

Figure Implementation.26: Implementation of the Random Forest model definition.

4.7.4 Support Vector Regression (SVR)

Support Vector Regression (SVR) was implemented as a scikit-learn pipeline consisting of a `StandardScaler` followed by an `SVR` model with a radial basis function (RBF) kernel. This

pipeline applies feature standardization automatically before fitting the model. Figure 4.16 shows the implementation of the SVR pipeline. The performance of the model is presented in Section 5.7.4.

```
414   svr_pipeline = Pipeline([("scaler", StandardScaler()),
415                            ("model", SVR(kernel="rbf"))])
416
```

Figure Implementation.27: Implementation of the Support Vector Regression (SVR) pipeline.

4.7.5 Long Short-Term Memory (LSTM)

The LSTM implementation begins by defining a sequence length of 30 operating cycles `(LSTM_WINDOW = 30).`

A separate `StandardScaler` is then fitted to the training features and used to standardize the predictor variables before sequence generation. The `build_sequences()` function is subsequently defined to convert the standardized engine data into fixed-length sliding-window sequences.

Each sequence contains 30 consecutive operating cycles and is paired with the corresponding target value before being passed to the LSTM model. Figure 4.17 shows the definition of the sequence length, feature scaling and the beginning of the sequence generation function.

```
564   LSTM_WINDOW = 30
565
566   # Scale features for LSTM (fitted on training engines only)
567   dev_scaler = StandardScaler().fit(X_train)
568   X_scaled = pd.DataFrame(dev_scaler.transform(X), columns=X.columns, index=X.index)
569
570
571   def build_sequences(features, target, engine_ids, cycles, engines, window,
572                       last_only=False):
573       """Build sliding-window sequences for LSTM input."""
574       seqs, targets, meta = [], [], []
```

Figure Implementation.28: Implementation of the sliding-window sequence generation for the LSTM.

The LSTM architecture was implemented using two LSTM layers with 64 and 32 units respectively, each followed by a dropout layer with a rate of 0.2.

The output from the LSTM layers was passed to a dense layer with 16 neurons using the `ReLU` activation function, followed by a final dense layer with a single output neuron for RUL prediction.

The model was compiled using the Adam optimizer and mean squared error (MSE) as the loss function. Training was controlled using an `EarlyStopping` callback that monitored the validation loss with a patience of five epochs and restored the weights from the best-performing epoch if no further improvement was observed.

A maximum of 50 training epochs was specified. Figure 4.18 shows the implementation of the LSTM architecture.

```
615  def build_lstm(n_features):
616      """Build a simple 2-layer LSTM model."""
617      model = Sequential([
618          LSTM(64, input_shape=(LSTM_WINDOW, n_features), return_sequences=True,
619              implementation=1),
620          Dropout(0.2),
621          LSTM(32, implementation=1),
622          Dropout(0.2),
623          Dense(16, activation="relu"),
624          Dense(1),
625      ])
626      model.compile(optimizer="adam", loss="mse")
627      return model
```

Figure Implementation.29: Implementation of the LSTM network architecture.

With all five models implemented, the next stage of the pipeline focused on data splitting, model training and validation.

4.8 Data Splitting and Cross-Validation Strategy

The engine identifiers were split into training and validation sets using `train_test_split()`, with all cycles belonging to the same engine assigned to the same dataset.

The resulting engine lists were then used to create the corresponding training and validation subsets for model development. Figure 4.19 shows the implementation of the engine-level data split.

```
367  all_engine_ids = train_data["engine_id"].unique()
368  train_ids, val_ids = train_test_split(all_engine_ids, test_size=TEST_SIZE,
369                                        random_state=SEED)
370
```

Figure Implementation.30: Implementation of the engine-level training and validation split.

Five-fold grouped cross-validation was implemented using GroupKFold, with the engine identifiers used as the grouping variable. The generated cross-validation folds were then used during model evaluation. Figure 4.20 shows the implementation of the grouped cross-validation procedure.

```
518   # --- 7c. Grouped cross-validation (all 100 engines) ------------------------
519   print("\n--- 5-Fold Grouped Cross-Validation ---")
520
521   cv_folds = list(GroupKFold(n_splits=N_FOLDS).split(X, y, groups=all_groups))
522   cv_results = []
```

Figure Implementation.31: Implementation of the 5-fold GroupKFold cross-validation.

4.9 Hyperparameter Optimization

Hyperparameter optimization for the Random Forest and Support Vector Regression (SVR) models was implemented using `RandomizedSearchCV`.

The predefined parameter search spaces, `GroupKFold` cross-validation strategy and RMSE scoring metric were supplied to the search procedure before fitting it to the training data.

For Random Forest, the search included the number of trees, maximum depth, minimum split size and minimum leaf size.

For SVR, the search included the penalty parameter (C), kernel width (gamma) and epsilon (ε). During development, the SVR search space was expanded after the initial search selected boundary values for C and gamma.

The final implementation therefore used the expanded SVR search space described in Section 3.9. Figure 4.21 shows the implementation of the hyperparameter optimization process.

```
447   rf_search = RandomizedSearchCV(rf_model, rf_params, n_iter=10,
448                                  cv=tuning_cv, scoring="neg_root_mean_squared_error",
449                                  random_state=SEED, n_jobs=-1)
450   rf_search.fit(X_train, y_train, groups=train_groups)
451   print(f"Best RF params: {rf_search.best_params_}")
452
453   # SVR tuning
454   svr_params = {
455       "model__C": [1, 10, 50, 100, 200, 500, 1000],
456       "model__gamma": ["scale", "auto", 0.001, 0.003, 0.01, 0.1],
457       "model__epsilon": [0.01, 0.1, 0.5, 1.0],
```

Figure Implementation.32: Implementation of hyperparameter optimization using RandomizedSearchCV

4.10 Final Model Retraining

After hyperparameter optimization, the best parameter values for Random Forest and SVR were retrieved and used to recreate the final models. All models were then trained using the complete set of 100 training engines.

The Linear Regression model was rebuilt as a pipeline containing a `StandardScaler` and the regression model, while the Random Forest and SVR models were rebuilt using their selected hyperparameters.

The LSTM model was retrained using the number of epochs determined during the validation stage.

Training time was recorded for each model before the final models were saved for evaluation on the official NASA test set. Figure 4.22 shows the implementation of the final retraining process.

```
670  # Retrain all models on the full 100 training engines
671  rf_best_params = rf_search.best_params_
672  svr_best_params = {k.replace("model__", ""): v
673                     for k, v in svr_search.best_params_.items()}
674
675  start = time.time()
676  final_lr = Pipeline([("scaler", StandardScaler()),
677                       ("model", LinearRegression())]).fit(X, y)
```

Figure Implementation.33: Implementation of the final model retraining

4.11 Model Evaluation

The official NASA test dataset was prepared by selecting the final operating cycle of each engine and extracting the corresponding predictor variables. The supplied RUL values were then loaded and capped at 125 cycles to match the training target before model evaluation. Figure 4.23 shows the implementation of the test dataset preparation.

```
728  test_last = test_data.groupby("engine_id").last().reset_index()
729  X_test = test_last[feature_cols]
730
731  # True RUL (capped for fair comparison)
732  true_rul = rul_data["true_rul"].values
733  true_rul_capped = np.clip(true_rul, 0, RUL_CAP)
734
```

Figure Implementation.34: Preparing the official test set: last cycle per engine and capped RUL.

For the LSTM model, the previously defined sequence generation function was reused to create the final sliding-window sequence for each test engine. The generated sequences were then used during the final model evaluation. Figure 4.24 shows the implementation of the LSTM test sequence generation.

```
739  X_test_seq, _, _ = build_sequences(
740      X_test_scaled, pd.Series(np.zeros(len(test_data)), index=test_data.index),
741      test_data["engine_id"], test_data["cycle"],
742      test_engine_ids, LSTM_WINDOW, last_only=True)
743
```

Figure Implementation.35: Implementation of the LSTM test sequence generation

4.12 Performance Evaluation Metrics

The performance of each model was evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), R^2 and the NASA PHM Score. The first three metrics were calculated using the corresponding scikit-learn functions, while the NASA PHM Score was implemented as a custom Python function. Figure 4.25 shows the implementation of the evaluation metrics.

```
752
753  # Evaluate all models
754  print("\n--- Test Results (capped RUL) ---")
755  test_results = []
756  for name, preds in predictions.items():
757      mae = mean_absolute_error(true_rul_capped, preds)
758      rmse = np.sqrt(mean_squared_error(true_rul_capped, preds))
759      r2 = r2_score(true_rul_capped, preds)
760      nasa_total, nasa_mean = nasa_phm_score(true_rul_capped, preds)
```

Figure Implementation.36: Implementation of the model evaluation metrics, including the NASA PHM Score.

In addition to the evaluation metrics, the script generated diagnostic plots for each model, including actual versus predicted RUL, residual plots and residual histograms. The final stage of the implementation focused on interpreting the best-performing model.

4.13 Model Interpretation and Diagnostic Analysis

Feature importance was extracted directly from the trained Random Forest model using its built-in impurity-based importance scores. The resulting importance values were saved and prepared for visualization. Figure 4.26 shows the implementation of the impurity-based feature importance.

```
828  # Impurity-based feature importance (Random Forest)
829  importance = pd.DataFrame({
830      "Feature": feature_cols,
831      "Importance": final_rf.feature_importances_,
```

Figure Implementation.37: Implementation of impurity-based feature importance for the Random Forest model.

Permutation importance was then calculated by repeatedly shuffling each predictor in the validation dataset and measuring its effect on model performance. The resulting importance scores were saved and prepared for comparison with the impurity-based importance values. Figure 4.27 shows the implementation of the permutation importance analysis.

```
838  # Permutation importance (on validation data)
839  perm = permutation_importance(rf_search.best_estimator_, X_val, y_val,
840                                n_repeats=10, random_state=SEED, n_jobs=-1)
841  perm_df = pd.DataFrame({
```

Figure Implementation.38: Implementation of the permutation importance analysis.

Both sets of importance scores were saved and visualized for comparison. The highest-ranked feature identified by the permutation importance analysis was then plotted across multiple engines for further inspection. The interpretation of these outputs is presented in Section 5.8.

4.14 Chapter Summary

This chapter has described how the methodology presented in Chapter 3 was implemented in Python. The implementation began with script initialization and data loading, followed by data preprocessing and exploratory data analysis.

The machine learning models were then implemented, validated, tuned and evaluated using the official NASA test dataset before the best-performing model was interpreted. All stages were completed within a single reproducible Python script, with the corresponding code presented throughout this chapter. The outputs generated by this implementation are presented and discussed in Chapter 5.

5 Results and Analysis

5.1 Introduction

This chapter presents the results obtained from implementing the machine learning models for turbofan remaining useful life (RUL) prediction. The results are presented in the same order as the implementation, beginning with data preparation and exploratory data analysis, followed by model development, hyperparameter optimization, validation and cross-validation, final retraining, evaluation on the official NASA test set, model interpretation and an overall comparison of the models. Each section presents the results, explains what they show, analyses their significance and evaluates their contribution to the overall performance of the prediction models.

5.2 Data Preparation Results

This section presents the results of the data preparation stage. The quality checks were used to confirm that the dataset was complete before feature selection. The results are presented below.

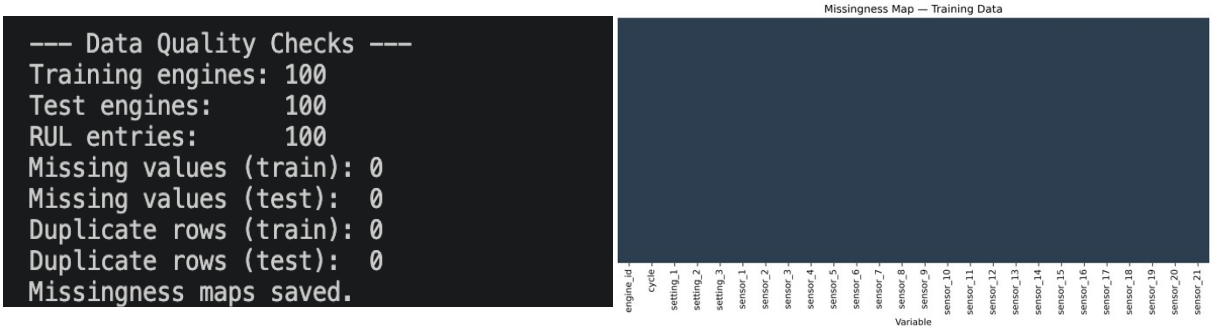


Figure Results and Analysis.39 Data quality checks. a: terminal output showing engine counts, RUL entries, missing values and duplicate records. b: missingness map for the training dataset.

This section presents the results of the data preparation stage. The quality checks were used to confirm that the dataset was complete before feature selection.

Figure 5.1 shows the results of the data quality checks. Figure 5.1(a) confirms that both the training and test datasets contain 100 engines, with 100 supplied RUL values for the test set. No missing values or duplicate records were found in either dataset. Figure 5.1(b) supports these findings by

showing no gaps across any variable in the training data. Together, these checks confirmed that the dataset was complete and ready for feature selection.

```
--- Removing Near-Constant Predictors ---
  Sensor and setting variability tables saved.
Sensors with std < 0.01: ['sensor_1', 'sensor_19', 'sensor_18', 'sensor_10', 'sensor_16', 'sensor_5', 'sensor_6']
Settings with a single distinct value: ['setting_3']
Remaining sensors: 14
Remaining settings: 2
Training data shape after removal: (20631, 18)
```

Figure Results and Analysis.40: Terminal output showing the results of the near-constant predictor removal process.

Figure 5.2 shows the results of the near-constant predictor checks. Seven sensors were removed because their standard deviations were below the selected threshold of 0.01. Setting 3 was also removed because it contained only one distinct value throughout the training dataset.

After feature selection, 14 sensors, 2 operational settings and the cycle variable remained, giving a total of 17 predictors. Including the RUL target, the processed training dataset contained 20,631 observations and 18 columns.

These results confirmed that the final predictor set was ready for exploratory data analysis and model development.

5.3 Exploratory Data Analysis Results

This section presents the results of the exploratory data analysis (EDA) conducted on the cleaned dataset before model training.

The analysis examined the distribution of the remaining useful life (RUL) target, assessed its normality, explored changes in sensor behaviour over time, and investigated the relationships between the retained predictors and RUL.

These results provided an initial understanding of the data and confirmed that the retained predictors contained information suitable for model development.

5.3.1 Distribution of the Target Variable

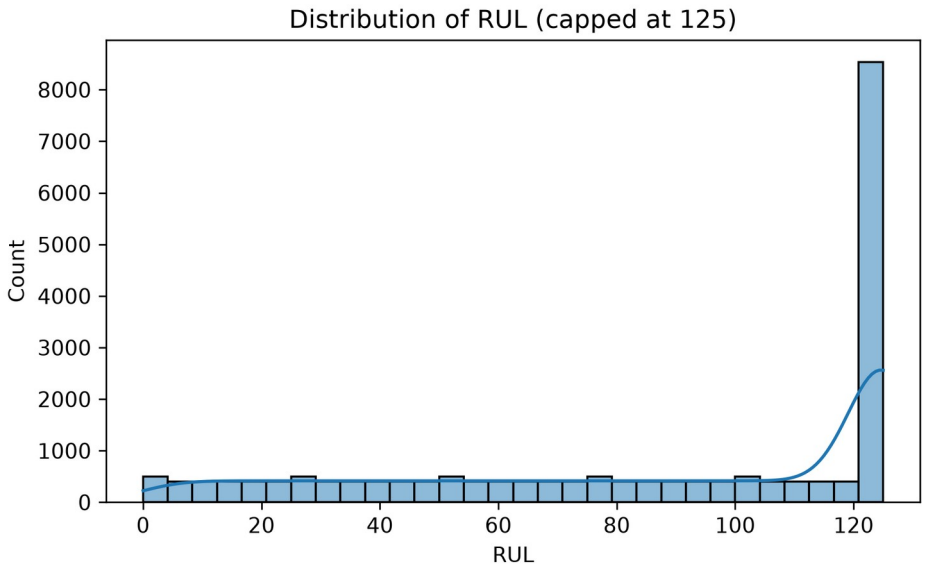


Figure Results and Analysis.41: Distribution of the capped RUL target across the training dataset.

Figure 5.3 shows the distribution of the capped RUL target in the training dataset. Most observations are concentrated at 125 cycles, while the remaining observations are distributed between 0 and 124 cycles.

This pattern resulted from the 125-cycle cap applied during data preparation, where all RUL values greater than 125 were assigned the same target value.

Consequently, most of the variation in the target occurs below 125 cycles, where the engines progress towards failure. Although the capped values account for a large proportion of the observations, the machine learning models still needed to learn the variation within the degradation stage rather than rely on predicting the dominant value.

For this reason, a Mean Baseline model was included later in the study to determine whether the learning models could capture this degradation pattern more effectively than a simple constant prediction.

5.3.2 Normality Assessment

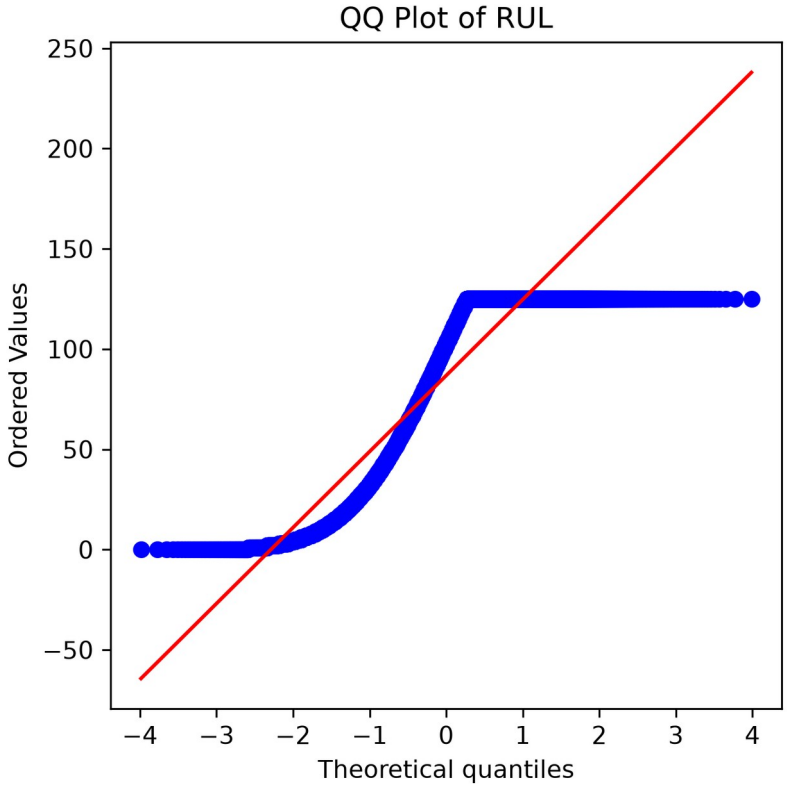


Figure Results and Analysis.42: Quantile–quantile (QQ) plot of the capped RUL target.

Figure 5.4 shows the QQ plot used to assess the normality of the RUL target. The data points deviate noticeably from the reference line, particularly at the lower and upper ends of the distribution.

The horizontal pattern at **125 cycles** reflects the RUL cap applied during data preparation, while the deviation at lower values indicates that the target does not follow a normal distribution.

This visual evidence suggests that the RUL target is non-normal and is confirmed by the statistical normality tests presented in Table 5.1.

Table Results and Analysis.5: Normality test results for the capped RUL target

Test	**Statistic**	**p-value**	**Decision on H_0**
Shapiro–Wilk	W = 0.821	< 0.001	Reject H_0
Kolmogorov–Smirnov	D = 0.214	< 0.001	Reject H_0

Table 5.1 confirms the visual assessment shown in Figure 5.4. Both the Shapiro–Wilk test (W = 0.821, $p < 0.001$) and the Kolmogorov–Smirnov test (D = 0.214, $p < 0.001$) rejected the null hypothesis of normality, confirming that the RUL target was not normally distributed.

This result was expected because the 125-cycle cap introduced a large concentration of observations at the upper limit of the distribution.

Although the RUL target was non-normal, this did not affect the machine learning models evaluated in this study, as none of the selected models require the target variable to be normally distributed.

Having examined the distribution of the target variable, the analysis next considers how the retained sensor measurements changed over an engine's operating life.

5.3.3 Sensor Trend Analysis

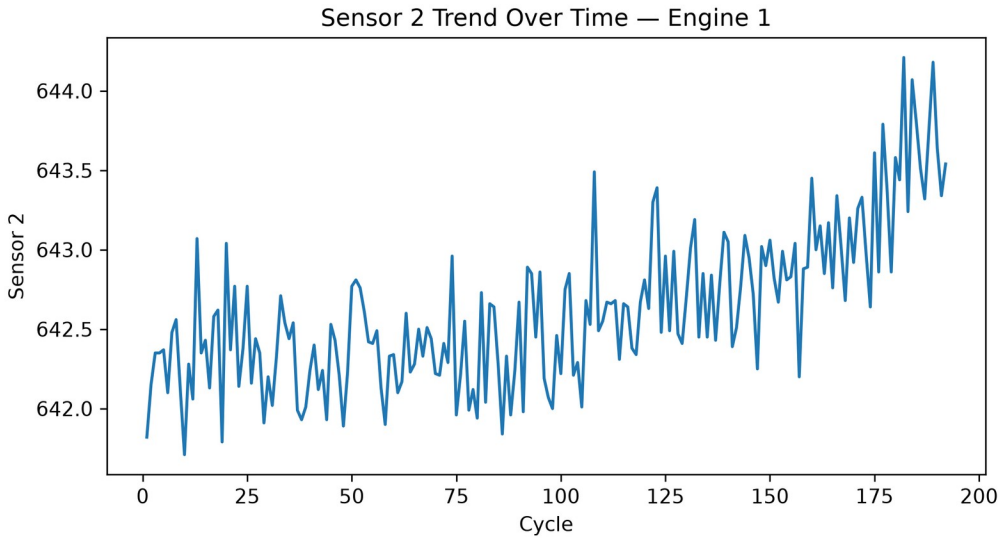


Figure Results and Analysis.43: Sensor 2 readings over the life of Engine 1

Figure 5.5 shows the behaviour of Sensor 2 over the operating life of Engine 1. During the early and middle cycles, the readings fluctuate within a relatively narrow range of approximately 642–643, with no sustained upward or downward trend.

After approximately cycle 150, however, the readings begin to increase more consistently, reaching their highest values during the final operating cycles.

This change suggests that Sensor 2 becomes more sensitive as the engine approaches failure, indicating that it may contain useful degradation information for predicting remaining useful life (RUL).

Since this figure represents only one sensor from a single engine, it cannot be used to assess the importance of predictors across the entire dataset.

A broader assessment of the relationships between all retained predictors and RUL is therefore presented in the following correlation analysis.

5.3.4 Correlation Analysis

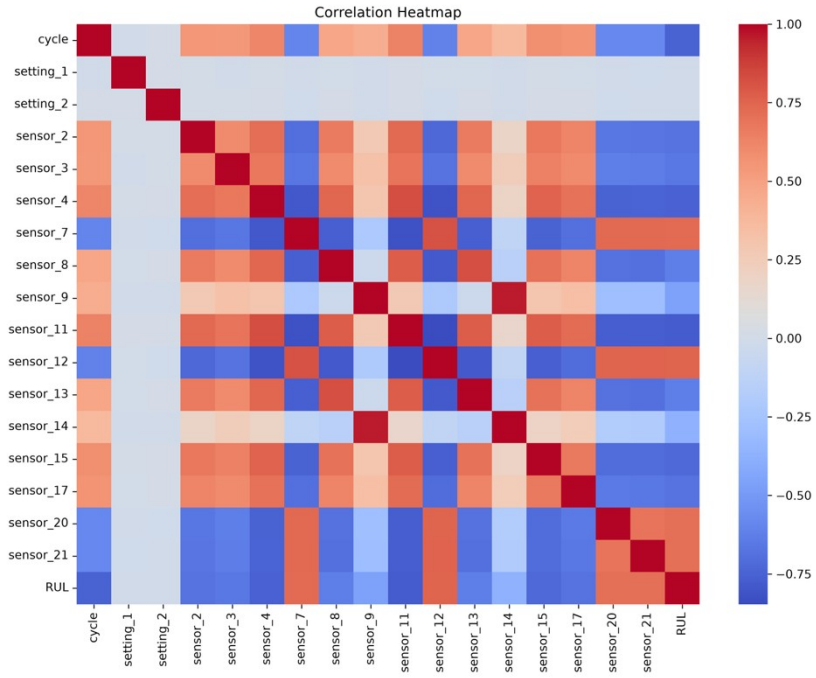


Figure Results and Analysis.44: Correlation heatmap of all 17 predictors and RUL

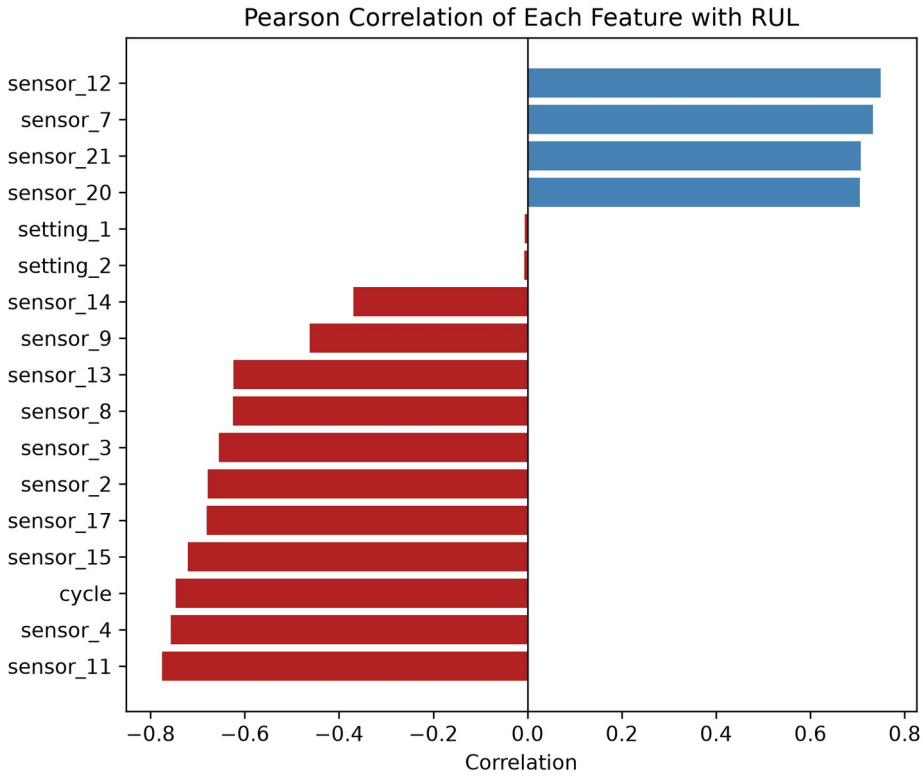


Figure Results and Analysis.45: Pearson correlation of each feature with RUL, ranked

Figures 5.6 and 5.7 present the relationships between the retained predictors and the RUL target. The correlation heatmap (Figure 5.6) shows clear groups of positively and negatively correlated sensors, indicating that several measurements change together as the engine degrades.

For example, Sensors 2, 3, 8, 13, 15 and 17 form one cluster with strong positive relationships, while Sensors 7, 12, 20 and 21 form another cluster with similar correlation patterns. This suggests that engine degradation is reflected across multiple sensors rather than a single measurement.

Figure 5.7 ranks the strength of the relationship between each predictor and RUL. Sensor 11 (−0.775), Sensor 4 (−0.757) and Cycle (−0.747) show the strongest negative correlations with RUL, indicating that their values change consistently as the remaining useful life decreases.

In contrast, Sensor 12 (0.749), Sensor 7 (0.733), Sensor 21 (0.707) and Sensor 20 (0.705) show the strongest positive correlations. Settings 1 and 2 have correlations close to zero (−0.006 and −0.007), indicating that they contribute very little linear information about RUL when considered individually.

Overall, these results show that the retained sensor measurements capture meaningful degradation patterns, providing a strong basis for the machine learning models developed in the next section.

5.4 Model Development

This section presents the results of the model development stage. Five prediction models were successfully implemented using the prepared training dataset: the Mean Baseline, Linear Regression, Random Forest, Support Vector Regression (SVR) and Long Short-Term Memory (LSTM).

Each model was developed using the implementation described in Chapter 4 to provide a consistent basis for comparison.

The Mean Baseline and Linear Regression required no automated hyperparameter optimization, while Random Forest and SVR proceeded to the optimization stage.

The LSTM architecture was predefined and therefore continued directly to validation. The results of the hyperparameter optimization are presented in the next section.

5.5 Hyperparameter Optimization Results

Table Results and Analysis.6: Best hyperparameters found for Random Forest and SVR

Model	**Selected Hyperparameters**
Random Forest	n_estimators=100, max_depth=10, min_samples_split=5, min_samples_leaf=1
SVR	C=500, gamma=0.01, epsilon=1.0

Table 5.2 presents the optimal hyperparameter configurations identified during the optimization process.

Hyperparameter optimization was performed only for Random Forest and SVR because these models contain several user-defined parameters that can substantially influence predictive performance.

Linear Regression has no meaningful hyperparameters requiring optimization, while the LSTM architecture and training settings were defined separately using the configuration described in Chapter 3.

For Random Forest, the selected configuration consisted of 100 trees, a maximum depth of 10, a minimum of 5 samples required for node splitting, and a minimum of 1 sample per leaf.

For SVR, the optimization selected C = 500, gamma = 0.01 and epsilon = 1.0. These parameter values were fixed for all subsequent validation, retraining and final evaluation to ensure that model comparisons were performed using the optimized configurations.

5.6 Validation and Cross-Validation Results

With the hyperparameters fixed, the five models were evaluated using the validation procedures described in Section 4.8.

The Mean Baseline, Linear Regression, Random Forest and Support Vector Regression (SVR) were assessed on the 20 held-out validation engines and further evaluated using five-fold grouped cross-validation.

The LSTM model was evaluated separately using its training and validation loss curves. The purpose of this stage was to determine whether each model could generalize to unseen engines before final retraining and evaluation on the official NASA test set.

5.6.1 Held-Out Validation

Table Results and Analysis.7: Performance of the five models on the 20 held-out validation engines.

Model	**MAE**	**RMSE**	**R^2**
Mean Baseline	36.93	41.72	−0.0002
Linear Regression	14.53	17.93	0.815
Random Forest	10.83	15.90	0.855
SVR	10.36	15.06	0.870
LSTM	10.04	13.26	0.899

Table 5.3 presents the performance of the five models on the 20 held-out validation engines. The LSTM achieved the strongest overall performance, recording the lowest MAE (10.04), lowest RMSE (13.26) and highest R^2 (0.899). Among the classical machine learning models, SVR ranked second with the lowest MAE (10.36), lowest RMSE (15.06) and highest R^2 (0.870), narrowly outperforming Random Forest. Linear Regression achieved lower predictive accuracy than both SVR and Random Forest.

The Mean Baseline produced an R^2 of −0.0002, indicating that it explained virtually none of the variation in RUL and performed marginally worse than simply predicting the average RUL. This confirms that a constant prediction cannot capture the engine-specific degradation patterns present in the dataset and provides an appropriate minimum benchmark against which the machine learning models can be compared.

Although the LSTM ranked first on the held-out validation set, the difference between SVR and Random Forest was relatively small. Because these results are based on only 20 held-out engines, the ranking should be interpreted cautiously. To determine whether the observed performance remained consistent across different groups of engines, the models were further evaluated using five-fold grouped cross-validation.

5.6.2 Overfitting Check

Table Results and Analysis.8: Training versus validation R^2 for each classical model.

Model	Training R^2	Validation R^2	Difference
Linear Regression	0.758	0.815	−0.057
Random Forest	0.904	0.855	+0.049
SVR	0.839	0.870	−0.031

Table 5.4 compares the training and held-out validation R^2 values for the classical machine learning models. Random Forest showed the largest reduction in performance, with R^2 decreasing from 0.904 on the training data to 0.855 on the validation engines.

Although this represents the greatest difference among the models, the reduction was modest and does not indicate substantial overfitting. Linear Regression and SVR achieved slightly higher R^2

values on the validation set than on the training data, suggesting that the held-out validation engines were marginally easier for these models rather than indicating improved learning.

The LSTM is not included in this comparison because its generalization was assessed using its training and validation loss curves rather than a direct comparison of training and validation R^2 values. This approach is more appropriate for neural networks, where model performance is monitored throughout the training process and the final model is selected using the validation loss.

Overall, the small differences between the training and validation results indicate that the classical machine learning models generalized well to unseen engines.

However, because these findings are based on a single held-out validation split, five-fold grouped cross-validation was performed to assess whether the observed performance remained consistent across different engine groupings.

5.6.3 Five-Fold Grouped Cross-Validation

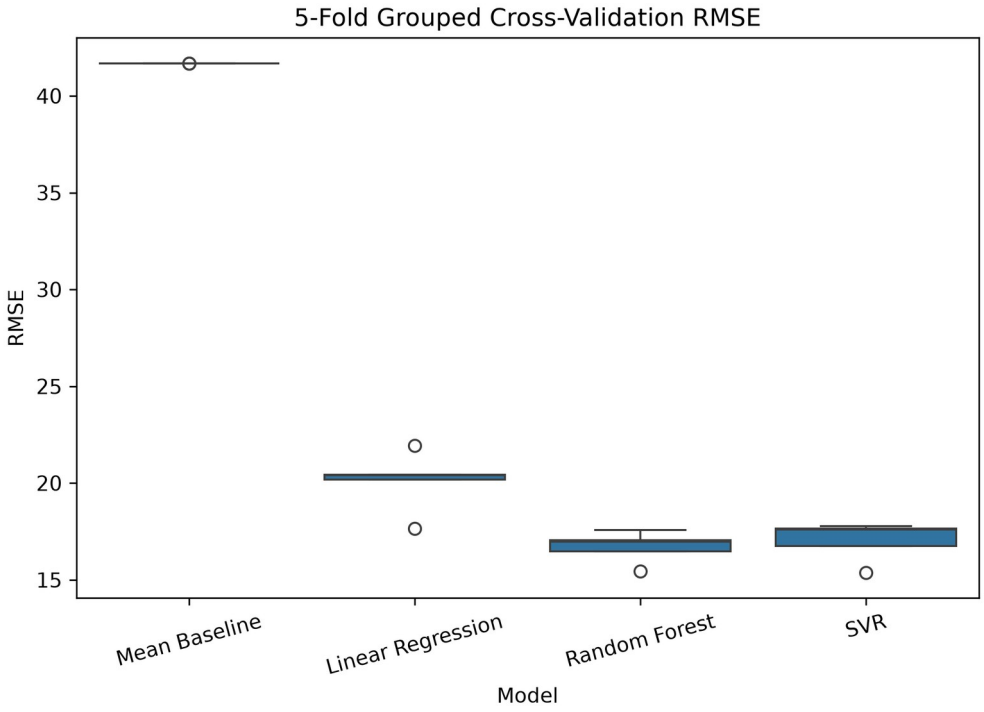


Figure Results and Analysis.46: RMSE across the five grouped cross-validation folds.

Table Results and Analysis.9: Average cross-validation performance across all five folds.

Model	Mean RMSE	RMSE Std	Mean R^2	R^2 Std
Mean Baseline	41.67	0.004	0.000	0.000
Linear Regression	20.12	1.547	0.766	0.035
Random Forest	16.70	0.807	0.839	0.015
SVR	16.99	0.998	0.833	0.019

Figure 5.8 and Table 5.5 present the results of the five-fold grouped cross-validation. Unlike the held-out validation, which evaluates performance on a single validation split, grouped cross-validation repeats the evaluation across five different engine groupings, providing a more robust assessment of model generalization.

Random Forest achieved the lowest mean RMSE (16.70) and the highest mean R^2 (0.839), with SVR producing very similar performance (mean RMSE 16.99 and mean R^2 0.833). Linear Regression produced larger prediction errors, while the Mean Baseline recorded the weakest performance across all folds.

The relatively small standard deviations for Random Forest (RMSE = 0.807; R^2 = 0.015) and SVR (RMSE = 0.998; R^2 = 0.019) indicate that both models performed consistently across the five validation folds. Figure 5.8 supports this finding, showing that the RMSE distributions for Random Forest and SVR overlap closely, whereas Linear Regression exhibited greater variation and the Mean Baseline remained substantially worse than all learning models.

Although the held-out validation results ranked SVR slightly ahead of Random Forest, grouped cross-validation placed Random Forest first. Given the small differences between the two models and the limited size of the held-out validation set, this change in ranking is not unexpected and suggests that their predictive performance is broadly comparable.

The LSTM was not included in the grouped cross-validation because the computational cost of retraining the sequence model and regenerating the sequence inputs for each fold was beyond the scope of this study. Its generalization was instead assessed using the held-out validation results and the training and validation loss curves presented in Section 5.6.4.

5.6.4 LSTM Training

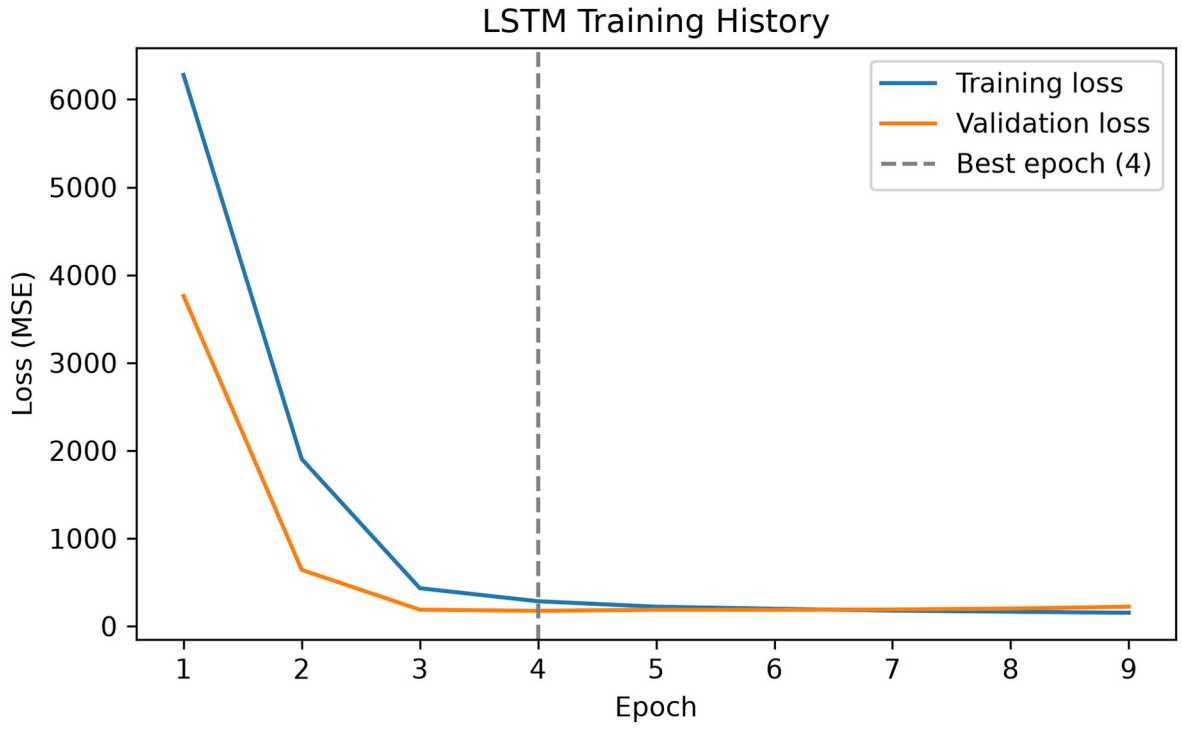


Figure Results and Analysis.47: LSTM training and validation loss across epochs

Figure 5.9 shows the training and validation loss curves for the LSTM model. Because the LSTM was not evaluated using the training–validation R^2 comparison or five-fold grouped cross-validation, these loss curves provide the primary evidence of its ability to generalize to unseen data. Both the training and validation loss decreased rapidly during the first four epochs, indicating that the model learned the underlying degradation patterns effectively.

The lowest validation loss was achieved at epoch 4. After this point, the training loss continued to decrease while the validation loss gradually increased, indicating the onset of overfitting. Early stopping terminated training after the patience limit was reached and restored the model weights from epoch 4, ensuring that the final model corresponded to the lowest validation loss rather than the final training epoch.

On the held-out validation set, the retained LSTM achieved an MAE of 10.04, an RMSE of 13.26 and an R^2 of 0.899, representing the strongest predictive performance among all evaluated models. However, unlike the classical machine learning models, the LSTM was not assessed using five-fold grouped cross-validation because retraining the sequence model and regenerating the sequence

inputs for each fold was beyond the computational scope of this study. The implications of this limitation are discussed in Chapter 6. These findings provided sufficient evidence to proceed to evaluation on the official NASA test set.

5.7 Official NASA Test Set Results

Following validation, the final model configurations were retrained using all 100 engines in the NASA FD001 training dataset and evaluated on the official NASA test set. Because none of the test engines were used during model development, this evaluation provides the final assessment of model performance on previously unseen engines.

Table Results and Analysis.10: Final performance of all five models on the official NASA test set (RUL capped at 125)

Model	**MAE**	**RMSE**	**R^2**	**NASA Score**
Mean Baseline	34.83	41.94	−0.095	33,354
Linear Regression	16.70	21.46	0.713	1,212
Random Forest	12.40	16.70	0.826	491
SVR	12.37	17.76	0.804	807
LSTM	10.42	14.60	0.867	314

Table 5.6 shows that the LSTM achieved the strongest overall predictive performance, recording the lowest MAE (10.42), lowest RMSE (14.60), lowest NASA score (314) and highest R^2 (0.867). These results indicate that the LSTM generalized most effectively to previously unseen engines.

Every machine learning model substantially outperformed the Mean Baseline, demonstrating that the models successfully learned meaningful relationships between the sensor measurements and Remaining Useful Life.

The Mean Baseline recorded an R^2 of −0.095, confirming that a constant prediction could not explain the variation in engine degradation.

Among the classical machine learning models, Random Forest achieved the strongest overall performance. Although SVR produced a marginally lower MAE (12.37 compared with 12.40), Random Forest achieved a lower RMSE (16.70), higher R^2 (0.826) and substantially lower NASA score (491 compared with 807), indicating fewer large prediction errors and stronger overall

predictive performance. Linear Regression outperformed the Mean Baseline but remained less accurate than the non-linear models across all evaluation metrics.

These results establish the LSTM as the best-performing model on the official NASA test set. The comparative performance of all five models and the practical implications of selecting the optimum model are examined in the following sections.

Table Results and Analysis.11: Training time comparison of the machine learning models (Times were measured on the same hardware and software environment and are intended for relative comparison within this study, not as universal benchmarks)

Model	**Training Time**	**Relative to Random Forest**
Linear Regression	< 0.1 s	faster
Random Forest	0.9 s	—
SVR	9.4 s	≈ 10× slower
LSTM	17.3 s	≈ 19× slower

Table 5.7 compares the computational cost of retraining the final models. Linear Regression required the shortest training time, completing in less than 0.1 seconds, while the LSTM required the longest at 17.3 seconds. Random Forest completed training in 0.9 seconds, making it substantially faster than both SVR (9.4 seconds) and the LSTM.

Although all models completed training in under 20 seconds on the 100-engine dataset, the relative differences in training time were considerable. Compared with Random Forest, SVR required approximately ten times longer to train, while the LSTM required approximately nineteen times longer. These results provide additional evidence for evaluating the trade-off between predictive performance and computational cost in the following sections.

5.7.1 Mean Baseline

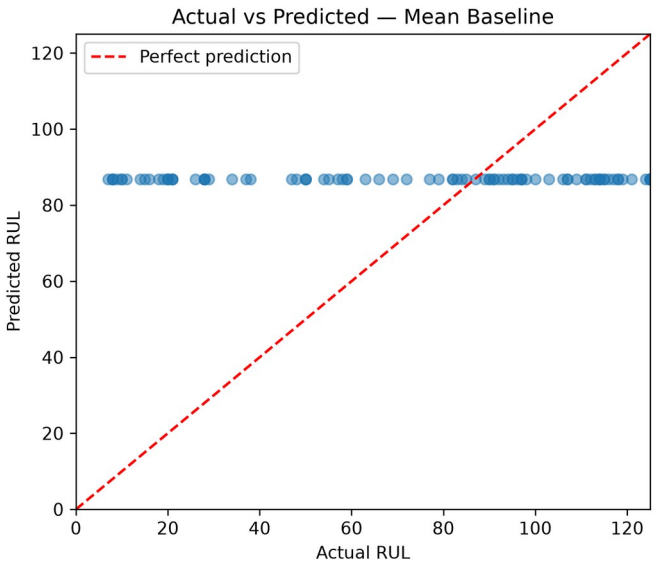


Figure Results and Analysis.48: Actual versus predicted RUL — Mean Baseline.

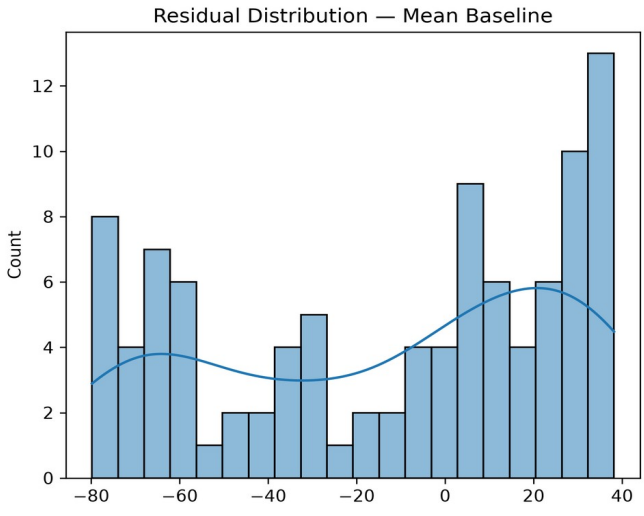


Figure Results and Analysis.49: Residual distribution — Mean Baseline.

Figures 5.10 and 5.11 show the prediction performance and residual distribution of the Mean Baseline model. As expected, the model predicted almost the same RUL value for every engine, producing a horizontal band rather than following the line of perfect prediction.

This resulted in large prediction errors across much of the RUL range. The residual distribution is widely spread and not concentrated around zero, indicating that the model consistently produced large over- and under-predictions.

These findings are reflected in Table 5.6, where the Mean Baseline recorded the highest MAE and RMSE, the lowest R^2, and the highest NASA score of all the models evaluated. The baseline therefore provides a useful reference point, demonstrating that the machine learning models learned meaningful degradation patterns beyond a simple constant prediction.

5.7.2 Linear Regression

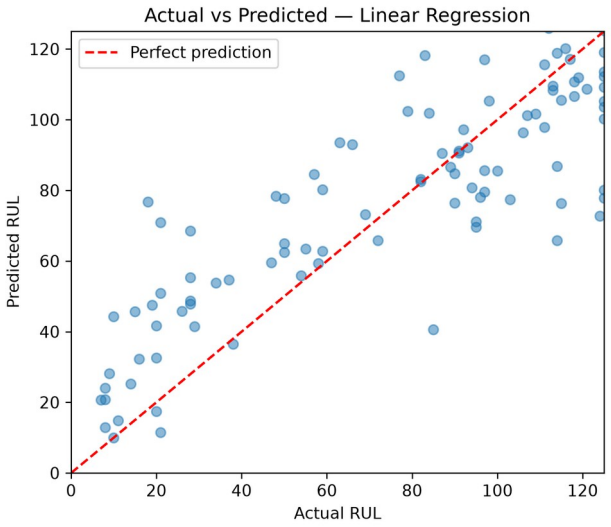


Figure Results and Analysis.50: Actual versus predicted RUL for the Linear Regression model

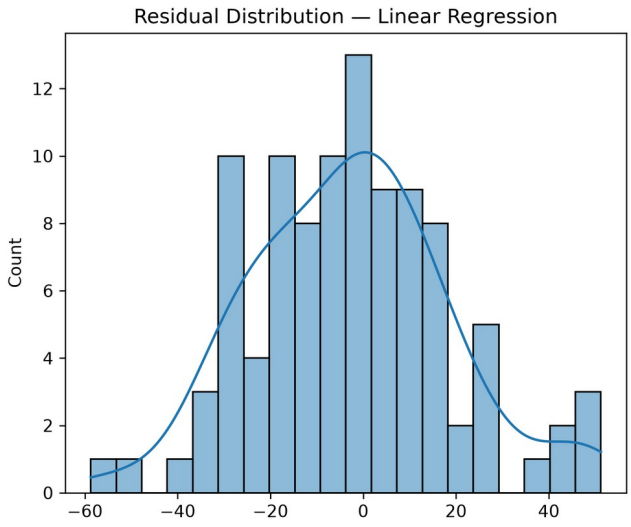


Figure Results and Analysis.51: Residual distribution for the Linear Regression model.

Figures 5.12 and 5.13 show the prediction performance and residual distribution of the Linear Regression model. The predicted values generally follow the line of perfect prediction but show considerable scatter across the RUL range.

The largest deviations occur at higher actual RUL values, where several predictions fall below the line of perfect prediction, indicating under-prediction for some healthier engines.

Figure 5.13 shows that the residuals are centered close to zero but remain widely distributed, with several large positive and negative errors.

These findings are consistent with the performance metrics reported in Table 5.6, where Linear Regression outperformed the Mean Baseline but remained less accurate than the non-linear machine learning models.

This suggests that the linear relationship assumed by the model was insufficient to capture the more complex degradation patterns present in the dataset.

5.7.3 Random Forest

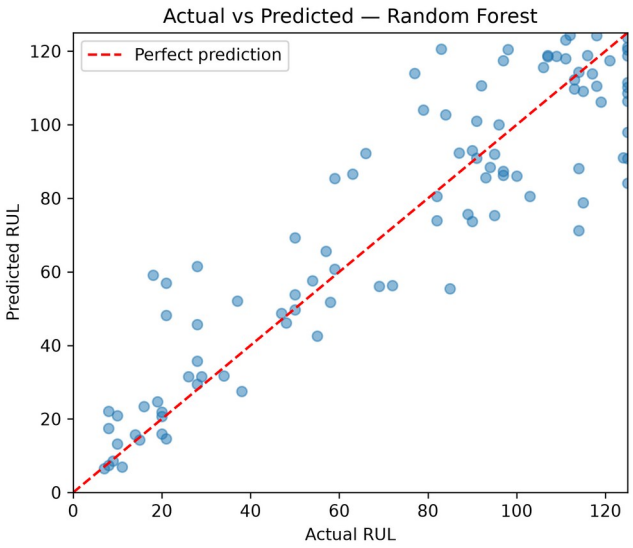


Figure Results and Analysis.52: Actual versus predicted RUL — Random Forest

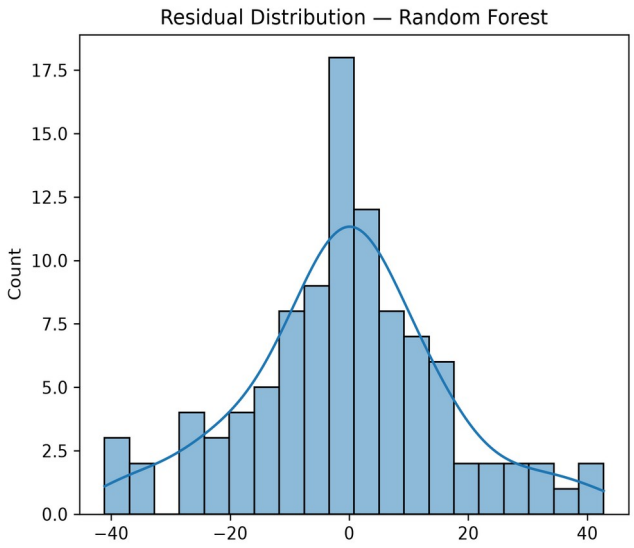


Figure Results and Analysis.53: Residual distribution — Random Forest

Figures 5.14 and 5.15 show the prediction performance and residual distribution of the Random Forest model. The predicted values follow the line of perfect prediction more closely than those of the Linear Regression model, indicating improved predictive accuracy across most of the RUL range.

Although some larger deviations remain, particularly at higher RUL values, the overall scatter around the diagonal is reduced.

Figure 5.15 shows that the residuals are concentrated around zero, with fewer extreme errors than those observed for Linear Regression.

These findings are consistent with the performance metrics reported in Table 5.6, where Random Forest achieved an MAE of 12.40, an RMSE of 16.70 and an R^2 of 0.826, making it the strongest-performing classical machine learning model on the official NASA test set.

The results suggest that the Random Forest model captured the non-linear relationships within the sensor data more effectively than the linear baseline.

5.7.4 Support Vector Regression (SVR)

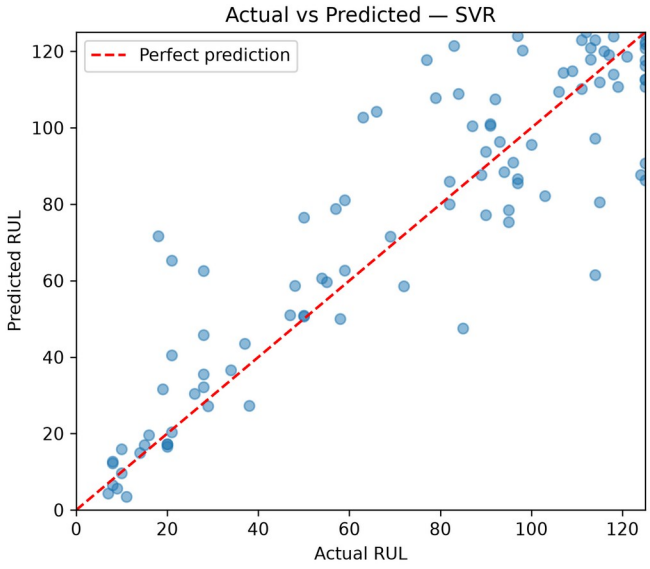


Figure Results and Analysis.54: Actual versus predicted RUL — SVR

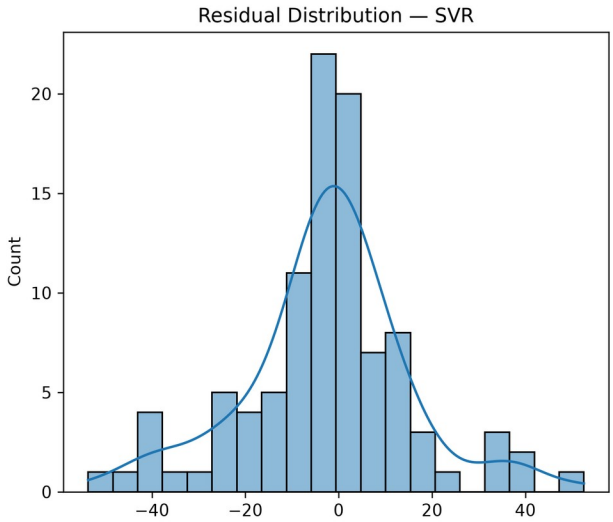


Figure Results and Analysis.55: Residual distribution — SVR

Figures 5.16 and 5.17 show the prediction performance and residual distribution of the Support Vector Regression (SVR) model.

The predicted values generally follow the line of perfect prediction across the RUL range, demonstrating that the model captured the overall degradation trend. However, larger deviations from the diagonal are visible for some engines, particularly at higher RUL values. Figure 5.17 shows that most residuals are concentrated around zero, although a small number of larger positive and negative residuals remain.

These findings are consistent with the performance metrics reported in Table 5.6, where SVR achieved an MAE of 12.37, an RMSE of 17.76 and an R^2 of 0.804. Although SVR produced a marginally lower MAE than Random Forest, its higher RMSE and NASA score indicate that it produced larger prediction errors for a smaller number of engines.

Overall, SVR performed strongly on the official NASA test set but remained slightly less consistent than Random Forest.

5.7.5 LSTM

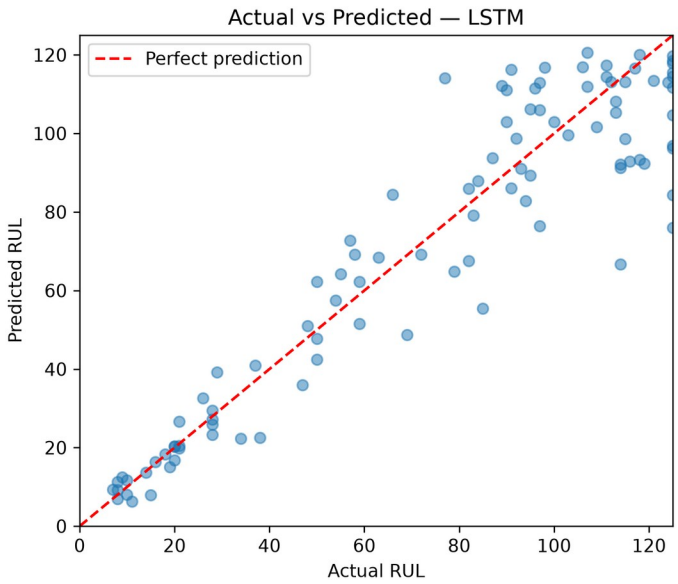


Figure Results and Analysis.56: Actual versus predicted RUL — LSTM

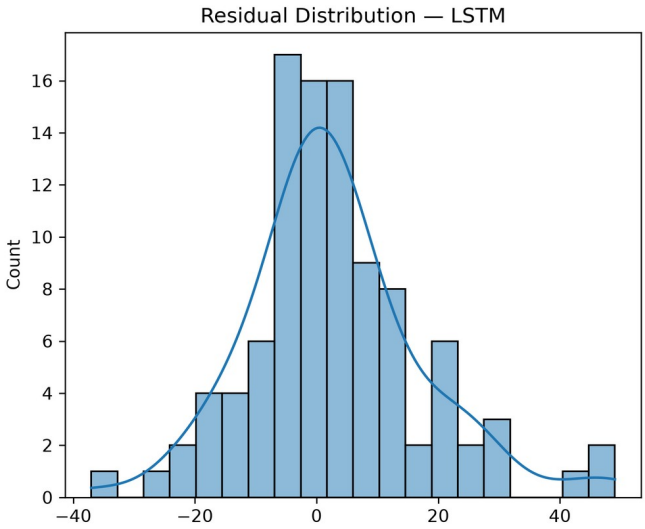


Figure Results and Analysis.57: Residual distribution — LSTM

Figures 5.18 and 5.19 show the prediction performance and residual distribution of the LSTM model. The predicted values follow the line of perfect prediction more closely than those of the other models, indicating that the LSTM captured the underlying degradation pattern with the highest level of accuracy.

Although some deviations remain, particularly at higher RUL values, the overall scatter around the diagonal is smaller than that observed for the classical machine learning models. Figure 5.19 shows that the residuals are concentrated around zero, with relatively few large positive or negative errors.

These observations are consistent with the performance metrics reported in Table 5.6, where the LSTM achieved the lowest MAE (10.42), lowest RMSE (14.60), lowest NASA score (314) and the highest R^2 (0.867) on the official NASA test set.

The results suggest that the LSTM benefited from modelling the sequential nature of the sensor data, enabling it to produce more accurate RUL predictions than the classical machine learning models.

5.8 Model Interpretation Results

This section examines the predictors that contributed most to the Random Forest model. Although the LSTM achieved the highest predictive performance, Random Forest was selected for interpretation because it was the strongest-performing classical machine learning model and provides feature importance measures that help explain its predictions.

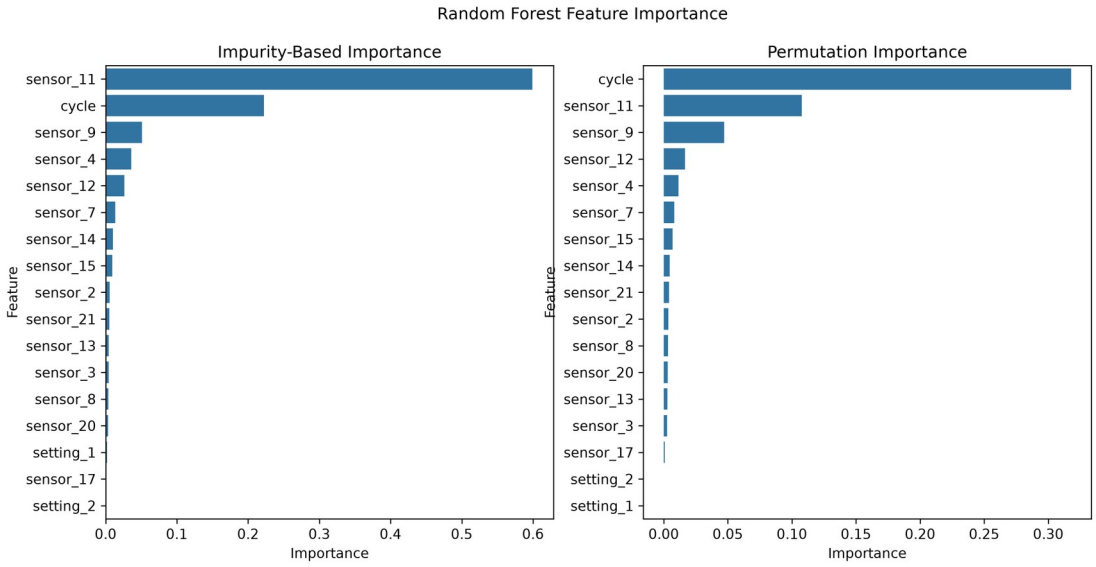


Figure Results and Analysis.58: Random Forest feature importance — impurity-based (left) and permutation (right)

Figure 5.20 presents the feature importance rankings produced by the Random Forest model using impurity-based and permutation importance measures.

Both methods identified cycle and sensor_11 as the two most influential predictors, although their order differed between the two approaches. Sensor_9, sensor_12 and sensor_4 were also consistently ranked among the most important features, while the operational settings contributed little to the model's predictions.

The agreement between the two methods indicates that the Random Forest model relied primarily on a small group of predictors when estimating remaining useful life.

The prominence of the cycle variable indicates that the model made substantial use of information about an engine's operating history, while the highly ranked sensor measurements show that it also captured changes associated with engine degradation.

5.9 Critical Evaluation of the Optimum Model

Table Results and Analysis.12: Summary comparison of model accuracy, computational cost and interpretability on the official NASA test set

Model	MAE (cycles)	RMSE (cycles)	R^2	NASA score	Training time
Mean Baseline	34.83	41.94	−0.095	33,354	—
Linear Regression	16.70	21.46	0.713	1,212	< 0.1 s
SVR	12.37	17.76	0.804	807	9.4 s
Random Forest	12.40	16.70	0.826	491	0.9 s
LSTM	**10.42**	**14.60**	**0.867**	**314**	**17.3 s**

The LSTM was selected as the optimum model because it achieved the lowest MAE and RMSE, the highest R^2 and the lowest NASA PHM score on the official test set. However, achieving the highest accuracy does not automatically make it the most appropriate model. The decision should also consider whether the improvement is practically meaningful and whether it justifies the additional computational cost and reduced interpretability.

Compared with Random Forest, the LSTM reduced MAE by 1.98 cycles and RMSE by 2.10 cycles. These improvements demonstrate superior predictive performance, but the margin is relatively small.

In contrast, the reduction in the NASA PHM score from 491 to 314 is more substantial, suggesting that the LSTM's greatest advantage lies in reducing late predictions, which are more costly in predictive maintenance. Therefore, the strongest justification for selecting the LSTM is not its lower average error, but its ability to reduce the types of errors that present the greatest operational risk.

The evidence supporting the LSTM should nevertheless be interpreted with caution. Unlike the classical machine learning models, it was not evaluated using grouped five-fold cross-validation because of the computational cost of retraining the sequence model for every fold. Its generalization was assessed using held-out validation, training and validation loss curves, and the official NASA test set. While these evaluations indicate strong predictive performance, they provide less evidence of stability across different engine groupings than is available for Random

Forest and SVR. Consequently, the superiority of the LSTM is supported by strong but comparatively narrower validation evidence.

The LSTM also depended on careful training to achieve its reported performance. Validation loss reached its minimum after four epochs before increasing, indicating the onset of overfitting. Early stopping restored the model weights from the best-performing epoch, preventing further deterioration in validation performance. This suggests that the reported accuracy reflects both the capability of the LSTM architecture and the effectiveness of the training strategy. Without appropriate regularization, the same level of performance may not have been achieved.

Random Forest represents the strongest alternative. Although it produced slightly higher prediction errors, it trained approximately nineteen times faster, provided direct feature importance measures and demonstrated consistent performance during grouped cross-validation. These characteristics make it easier to interpret and provide stronger evidence of robustness across different validation splits. Therefore, if computational efficiency and model transparency are considered alongside predictive accuracy, Random Forest offers a compelling practical alternative to the LSTM.

Overall, the LSTM is the optimum model within the scope of this study because predictive accuracy was the primary evaluation criterion and it consistently achieved the strongest results on the official NASA test set. However, its superiority should not be overstated. The performance improvement over Random Forest was modest, supported by less extensive validation evidence and achieved at the expense of greater computational cost and lower interpretability.

Consequently, while the LSTM is the preferred model where maximum predictive accuracy is required, Random Forest remains a highly competitive choice where transparency, computational efficiency and practical deployment are equally important.

5.10 Chapter Summary

This chapter presented the results of the data preparation, exploratory data analysis, model development and model evaluation. The findings demonstrated that the retained predictors contained sufficient information to support accurate Remaining Useful Life prediction and that all machine learning models outperformed the Mean Baseline on previously unseen engines. Among

the models evaluated, the LSTM achieved the strongest overall predictive performance on the official NASA test set, while Random Forest emerged as the strongest classical machine learning model by providing competitive predictive accuracy alongside lower computational cost and greater interpretability.

The critical evaluation showed that although the LSTM was selected as the optimum model, its advantage over Random Forest was relatively modest and should be considered alongside the trade-offs in computational efficiency, interpretability and validation evidence.

The next chapter discusses these findings in relation to previous research, considers their practical implications, and examines the limitations of the study and recommendations for future work.

6 Critical Evaluation, Discussion and Conclusion

6.1 Introduction

This chapter critically evaluates the research process and brings together the dissertation's main conclusions. It assesses the research design, methodology, model development and evaluation strategy against the stated aim and research questions. It also reflects on important research decisions, discusses the findings in relation to previous work, identifies limitations and contributions, and presents practical recommendations and future research directions.

6.2 Critical Evaluation of the Research Process

6.2.1 Critical Evaluation of the Research Design

The quantitative, supervised and comparative experimental design was appropriate for evaluating machine learning models for turbofan Remaining Useful Life (RUL) prediction. Using FD001, common preprocessing principles and the same final test set created a controlled basis for comparison. This addressed a problem identified in the literature: model results are difficult to compare when datasets, preprocessing and evaluation differ (Lei et al., 2018).

The main strength of the design was experimental control. FD001 contains one operating condition and one fault mode, reducing variation from different operating regimes and making model differences easier to interpret. However, this also limited external validity. FD001 is simulated and less complex than FD002-FD004 or real operational engine data (Saxena and Goebel, 2008). The findings therefore support comparative performance under FD001 conditions, not universal model rankings across real fleets.

Alternative designs were possible. Testing FD001-FD004 or real industrial data would improve generalizability but introduce greater operating-condition and fault-mode variation. Repeated engine-level splits could also quantify performance variability more directly. For an MSc comparison, the chosen design reasonably prioritized experimental control and feasibility over broad generalization.

6.2.2 Critical Evaluation of the Methodological Implementation

The implementation closely followed Chapter 3: data loading, quality checks, RUL construction, feature selection, exploratory analysis, engine-level splitting, model development, optimization, validation, final retraining and official test evaluation. This alignment made the research traceable and reduced the risk of selecting methods only after seeing the results.

Several preprocessing choices were appropriate. Near-constant variables were removed, leaving 17 predictors. Standardization was applied only to scale-sensitive models, and the scaler was fitted on training data rather than validation or test data. The 80:20 split and GroupKFold were both engine-based, so cycles from one engine could not appear in both training and validation. This is more defensible than a random row-level split, which could produce optimistic results through leakage.

Not every analytical step contributed equally to model validity. Normality testing of the capped RUL target was mainly descriptive because the selected models do not require a normally distributed target. Greater value could have come from sensitivity analysis of the 125-cycle RUL cap or an ablation study comparing performance before and after feature removal. The near-constant threshold should also be treated as an FD001-specific practical rule rather than a universal criterion.

Overall, the strongest methodological features were leakage-safe engine-level validation, model-specific scaling and strict separation of the official test set. The main weakness was limited sensitivity analysis of preprocessing choices and no formal ablation study showing how much individual decisions affected performance.

6.2.3 Critical Evaluation of Model Development and Evaluation Validity

The model set covered increasing levels of complexity: Mean Baseline, Linear Regression, Random Forest, SVR and LSTM. This allowed comparison of a non-learning reference, a linear benchmark, non-linear classical models and a sequence-based deep learning model while retaining a manageable scope.

Development effort was not fully symmetrical. Random Forest and SVR were optimized using RandomizedSearchCV with GroupKFold, whereas the LSTM used a fixed architecture with early stopping. This was practical, but different sequence lengths, layer sizes or learning rates might have changed the LSTM result. Its lead therefore applies to the implemented configuration, not to LSTM models generally.

The SVR optimization process showed why automated search still requires judgement. The initial search selected C and gamma at the boundaries of the search space, suggesting that useful values might lie outside it. The range was widened and the final parameters were selected from within the expanded space. This strengthened the process and showed that optimization quality depends partly on the search space supplied (Bergstra and Bengio, 2012).

Evaluation validity was strengthened by using MAE, RMSE, R^2 and the NASA PHM score. These captured average error, large errors, explained variation and risk-sensitive late predictions. The fact that Random Forest and SVR changed order across some measures shows why this multi-metric approach mattered. Held-out validation, grouped cross-validation and the official NASA test set also reduced dependence on one development split.

The main weakness was that grouped cross-validation was not performed for the LSTM. Its evidence came from held-out validation, loss curves and the official test set. Validation loss was lowest at epoch 4 and then increased, so early stopping materially affected the retained model. The study also lacked confidence intervals, significance tests and repeated neural-network runs. Large gains over the baseline are persuasive, but small Random Forest/SVR differences should not be treated as statistically proven. The evaluation was therefore strong for broad patterns but weaker for fine-grained ranking claims.

6.3 Dissertation Diary and Critical Reflection

The research process changed my understanding of what makes a machine learning comparison reliable. At the beginning, the most visible task was building the models. As the work developed, it became clear that the more important issue was how the data were separated and evaluated. The 20,631 training records represented repeated operating cycles from only 100 engines, not 20,631 independent cases. Recognizing this changed how I understood the unit of analysis and led to

greater attention to engine-level splitting and grouped validation. The main lesson was that a model can produce impressive numbers and still be evaluated incorrectly if related observations are allowed to cross between training and validation data.

A second important stage involved the order of feature selection, splitting and scaling. Working through these steps clarified that the scaler for scale-sensitive models had to be fitted only on training data. This strengthened my understanding that preprocessing is part of model evaluation, not simply a separate cleaning stage.

Hyperparameter optimization also changed my approach. When the first SVR search selected values at the edge of the parameter range, accepting the output automatically would have been easy. Expanding the search space showed that optimization results still need to be questioned. I learned to treat a boundary solution as a sign that the search design itself may need revision.

The LSTM produced a different challenge. Reproducibility required fixed seeds, and training had to be judged from validation behaviour rather than the final epoch. Validation loss was lowest at epoch 4 and then rose, so early stopping prevented deterioration. This showed me that model complexity creates additional decisions that can directly affect the reported result. The wider lesson was that credible machine learning research depends on the complete process, not only the final ranking.

6.4 Discussion and Synthesis of the Findings

The three FD001 files served different stages of the experiment. The training file contained complete run-to-failure trajectories for 100 engines and was used for RUL construction, model development and validation. The separate test file contained 100 different engines whose trajectories ended before failure, while the supplied RUL file provided the ground-truth remaining life for those test engines. Keeping model development within the training engines and reserving the official test engines for final assessment helped separate model selection from final evaluation (Saxena and Goebel, 2008).

The results show that the FD001 sensor data contained substantial predictive information beyond a constant RUL estimate. On the official NASA test set, the Mean Baseline produced an MAE of

34.83 cycles, RMSE of 41.94 and R^2 of -0.095. A negative R^2 is possible when predictions perform worse than simply using the mean of the observed test outcomes as a reference. Here, the constant training-mean prediction provided little useful information about differences in RUL between unseen engines; the negative value is therefore not a metric error, but evidence of a weak baseline. In contrast, all four learning models substantially reduced error. The LSTM achieved MAE of 10.42 cycles, RMSE of 14.60, R^2 of 0.867 and the lowest NASA score of 314, with an MAE about 70% lower than the baseline.

The LSTM was the strongest overall model, but the size and nature of its advantage are more informative than the ranking alone. Compared with Random Forest, the LSTM reduced MAE from 12.40 to 10.42 cycles, an improvement of about 16%, and reduced RMSE from 16.70 to 14.60, about 12.6%. The difference was larger on the NASA score, which fell from 491 to 314, an improvement of about 36%. This matters because the NASA metric gives greater weight to predictions that overestimate remaining life. The LSTM therefore showed its clearest advantage on the error pattern that is more concerning for maintenance risk, not simply on average error.

That advantage came with a computational and interpretability trade-off. The LSTM, which represented deep learning in this study, required about 17.3 seconds to train compared with 0.9 seconds for Random Forest, making it roughly nineteen times slower. However, the relative difference should be interpreted in context because every model still trained in under 20 seconds on FD001. The computational disadvantage would become more important if deep learning training were repeated across multiple random seeds, hyperparameter combinations, cross-validation folds or larger datasets. Random Forest also provided direct feature-importance measures, whereas the LSTM was not interpreted directly. The LSTM therefore offered better prediction at the cost of greater training and validation effort and lower transparency, consistent with the broader trade-offs reported for deep learning models (Zheng et al., 2017; Goodfellow, Bengio and Courville, 2016).

The comparison between Random Forest and SVR provides a second distinctive finding. On the official test set, SVR had a marginally lower MAE than Random Forest, 12.37 compared with 12.40 cycles. However, Random Forest had lower RMSE (16.70 versus 17.76), higher R^2 (0.826 versus 0.804) and a much lower NASA score (491 versus 807). The validation stages also changed their ordering: SVR was slightly stronger on the single held-out set, whereas five-fold grouped

cross-validation placed Random Forest ahead with mean RMSE of 16.70 compared with 16.99 for SVR. This shows why selecting a model from one split or one metric can lead to a different conclusion. The most defensible interpretation is that Random Forest and SVR were broadly competitive, with Random Forest providing the stronger overall evidence when error severity, grouped validation and interpretability are considered together.

Linear Regression remains important because its test R^2 of 0.713 was far above the baseline but below the non-linear models. This suggests that part of the RUL relationship is approximately linear, while the additional gains from Random Forest, SVR and LSTM indicate that non-linear and temporal structure matters, consistent with previous RUL research (Lei et al., 2018; Zheng et al., 2017).

Model interpretation added a final qualification. Random Forest importance measures identified cycle and sensor_11 as the two most influential predictors, with sensor_9, sensor_12 and sensor_4 also important. The strong role of cycle indicates that the model used elapsed operating age as a major predictive signal alongside degradation sensors. Cycle is available at prediction time and is therefore not direct target leakage, but its importance means the model's performance should not be interpreted as arising from sensor condition alone. A useful future test would therefore compare models with and without cycle to quantify how much predictive performance depends on this age prior.

Taken together, the findings broadly support the expectations established in Chapter 2. Classical machine learning offered competitive prediction with lower computational cost and greater transparency, while the sequence-based LSTM achieved the strongest predictive performance by modelling temporal degradation. At the same time, the changing Random Forest/SVR ordering across metrics and validation stages supports the literature's warning that no model is universally superior. Direct numerical comparison with published studies should therefore remain cautious because RUL definitions, preprocessing, model configurations and evaluation protocols differ between studies (Lei et al., 2018).

6.5 Conclusions in Relation to the Aim and Research Questions

The study achieved its aim of evaluating and comparing selected machine learning models for turbofan RUL prediction using NASA C-MAPSS FD001. Table 6.1 gives a direct evidence-based answer to each research question.

Table Critical Evaluation, Discussion and Conclusion.13: Summary of research questions, findings and conclusions

RQ	Question	Main finding	Conclusion
RQ1	What does the literature reveal?	ML is well established for RUL prediction, but methodological differences make published results difficult to compare directly.	A controlled comparison under common conditions was justified.
RQ2	What methodology is appropriate?	Engine-level splitting, leakage-safe scaling, grouped validation and a reserved NASA test set produced a suitable framework.	The methodology was appropriate, although sensitivity analysis and fully symmetrical validation could improve it.
RQ3	How effectively can the models predict RUL?	All learning models beat the baseline; LSTM achieved MAE 10.42, RMSE 14.60 and R^2 0.867.	The selected machine learning models predicted FD001 RUL effectively, with LSTM strongest.
RQ4	How do the models compare?	LSTM was strongest overall. Random Forest/SVR ordering changed across held-out validation, grouped CV and individual metrics.	Model ranking depends on both the evaluation metric and the validation evidence.
RQ5	What are the strengths, limitations and implications?	LSTM led prediction and NASA score; Random Forest offered lower cost and direct interpretation. FD001 and unequal validation depth limit generalization.	The most suitable model depends on operational priorities rather than predictive accuracy alone.

6.6 Limitations

Four limitations define the boundaries of the conclusions. First, the study used only FD001, which is simulated and contains one operating condition and one fault mode. This strengthened experimental control but limits external validity; the observed model ranking cannot be assumed to transfer to FD002–FD004 or real aircraft engines.

Second, the model-development and validation effort was not fully symmetrical. Random Forest and SVR received grouped hyperparameter optimization and five-fold grouped cross-validation, whereas the LSTM used a fixed architecture, early stopping and no grouped cross-validation. Its strong official-test performance is therefore credible within the implemented experiment, but its stability is supported by narrower evidence.

Third, the experiment did not include repeated runs across different neural-network seeds, confidence intervals or formal statistical tests of model differences. This matters most for close comparisons such as Random Forest versus SVR. Their small error differences should be interpreted as broadly comparable performance rather than definitive statistical superiority.

Finally, model interpretation was performed only for Random Forest. The importance of cycle and sensor_11 cannot be assumed to explain the behaviour of SVR or LSTM. These limitations do not overturn the large improvement of the learning models over the baseline, but they reduce confidence in claims about exact model ordering, generalization to real fleets and the reasons behind the LSTM's predictions.

6.7 Future Direction

6.7.1 Practical Recommendations

For an FD001-type RUL task, the LSTM is the preferred option when maximum predictive performance and reduction of risk-sensitive error are the main priorities. Random Forest is a strong alternative when training speed, transparency and simpler implementation are important.

In either case, model predictions should support maintenance judgement rather than automatically determine maintenance action. Before operational use, the selected model should be validated on representative real or more varied engine data, and performance should be monitored using both conventional error metrics and an asymmetric measure such as the NASA PHM score.

6.7.2 Academic Contribution

The contribution of this dissertation is methodological and empirical rather than a new algorithm. It provides a controlled comparison of simple, classical and sequential models using engine-level separation, leakage-safe scaling and a reserved official test set.

It also demonstrates that the meaning of 'best model' depends on both metric and validation stage: SVR slightly led Random Forest on held-out validation, while grouped cross-validation and most official-test measures favoured Random Forest.

The LSTM's gain over Random Forest was modest on MAE and RMSE but larger on the risk-sensitive NASA score, while requiring about nineteen times the training time. The study therefore contributes evidence that headline accuracy alone is an incomplete basis for model selection.

6.7.3 Future Research

Future research should first test whether the findings persist on FD002–FD004 and on real operational data. The LSTM should be evaluated across repeated random seeds and, where computationally feasible, within an engine-grouped cross-validation framework so that its stability can be compared more directly with the classical models.

A systematic LSTM hyperparameter search would also make the development effort more comparable with Random Forest and SVR. Additional models such as XGBoost could broaden the classical comparison, but they should be added without weakening the consistency of the experimental framework.

Finally, explainability should be extended beyond Random Forest, and an ablation experiment should remove cycle to test how strongly model performance depends on elapsed-age information rather than sensor degradation alone.

6.8 Overall Conclusion

This dissertation set out to evaluate and compare machine learning models for turbofan RUL prediction using NASA C-MAPSS FD001 under a consistent experimental framework. One hundred run-to-failure training engines supported model development and validation, while 100 separate official test engines provided the final comparison.

The LSTM produced the strongest test performance, achieving an MAE of 10.42 cycles, an RMSE of 14.60, an R^2 of 0.867 - explaining 86.7% of the observed test-RUL variation - and the lowest NASA PHM score of 314.

Random Forest remained a strong practical alternative because it trained about nineteen times faster, was directly interpretable and had stronger grouped-validation evidence. The study therefore does not establish a universal best model.

Within FD001, LSTM was the strongest predictive model, while the wider conclusion is that RUL model selection should balance predictive error, maintenance risk, computational cost, interpretability and the strength of validation evidence.

References

Asif, O., Haider, S.A., Naqvi, S.R., Zaki, J.F.W., Kwak, K.-S. and Islam, S.M.R. (2022) 'A deep learning model for remaining useful life prediction of aircraft turbofan engine on C-MAPSS dataset', *IEEE Access*, 10, pp. 95425–95440. Available at: https://doi.org/10.1109/ACCESS.2022.3203406.

Babu, G.S., Zhao, P. and Li, X.-L. (2016) 'Deep convolutional neural network-based regression approach for estimation of remaining useful life', in S.B. Navathe et al. (eds.) *Database Systems for Advanced Applications (DASFAA 2016)*. Cham: Springer, pp. 214–228. Available at: https://doi.org/10.1007/978-3-319-32025-0_14.

Bergstra, J. and Bengio, Y. (2012) 'Random search for hyper-parameter optimization', Journal of Machine Learning Research, 13, pp. 281–305.

Bishop, C.M. (2006) Pattern recognition and machine learning. New York: Springer.

Breiman, L. (2001) 'Random forests', Machine Learning, 45(1), pp. 5–32. Available at: https://doi.org/10.1023/A:1010933404324.

Chapelle, O., Schölkopf, B. and Zien, A. (eds.) (2006) Semi-supervised learning. Cambridge, MA: MIT Press.

Chen, T. and Guestrin, C. (2016) 'XGBoost: A scalable tree boosting system', in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. San Francisco, CA: ACM, pp. 785–794. Available at: https://doi.org/10.1145/2939672.2939785.

Dida, M., Cheriet, A. and Belhadj, M. (2025) 'Remaining useful life prediction using Attention-LSTM neural network of aircraft engines', International Journal of Prognostics and Health Management, 16(2). Available at: https://doi.org/10.36001/ijphm.2025.v16i2.4274.

Ellefsen, A.L., Bjørlykhaug, E., Æsøy, V., Ushakov, S. and Zhang, H. (2019) 'Remaining useful life predictions for turbofan engine degradation using semi-supervised deep architecture', Reliability Engineering & System Safety, 183, pp. 240–251. Available at: https://doi.org/10.1016/j.ress.2018.11.027.

Elsherif, S.M., Hafiz, B., Makhlouf, M.A. and Farouk, O. (2025) 'A deep learning-based prognostic approach for predicting turbofan engine degradation and remaining useful life', Scientific Reports, 15, article 26251. Available at: https://doi.org/10.1038/s41598-025-09155-z.

Ensarioğlu, K., İnkaya, T. and Emel, E. (2023) 'Remaining useful life estimation of turbofan engines with deep learning using change-point detection-based labeling and feature engineering', *Applied Sciences*, 13(21), 11893. Available at: https://doi.org/10.3390/app132111893.

Field, A. (2018) *Discovering statistics using IBM SPSS Statistics*. 5th edn. London: Sage.

Ghasemi, A. and Zahediasl, S. (2012) 'Normality tests for statistical analysis: a guide for non-statisticians', *International Journal of Endocrinology and Metabolism*, 10(2), pp. 486–489. Available at: https://doi.org/10.5812/ijem.3505.

Goodfellow, I., Bengio, Y. and Courville, A. (2016) *Deep learning*. Cambridge, MA: MIT Press.

Han, J., Kamber, M. and Pei, J. (2012) *Data mining: concepts and techniques*. 3rd edn. Waltham, MA: Morgan Kaufmann.

Heimes, F.O. (2008) 'Recurrent neural networks for remaining useful life estimation', in *2008 International Conference on Prognostics and Health Management*. Denver, CO, 6–9 October. Piscataway, NJ: IEEE, pp. 1–6. Available at: https://doi.org/10.1109/PHM.2008.4711422.

Hochreiter, S. and Schmidhuber, J. (1997) 'Long short-term memory', *Neural Computation*, 9(8), pp. 1735–1780. Available at: https://doi.org/10.1162/neco.1997.9.8.1735.

Hou, G., Xu, S., Zhou, N., Yang, L. and Fu, Q. (2020) 'Remaining useful life estimation using deep convolutional generative adversarial networks based on an autoencoder scheme', *Computational Intelligence and Neuroscience*, 2020, article 9601389. Available at: https://doi.org/10.1155/2020/9601389.

Jagannath, V. (2017) *Random forest diagram complete* [Image]. Wikimedia Commons. Available at: https://commons.wikimedia.org/wiki/File:Random_forest_diagram_complete.png (Accessed: 23 August 2026).

James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021) *An introduction to statistical learning with applications in R*. 2nd edn. New York: Springer.

Jardine, A.K.S., Lin, D. and Banjevic, D. (2006) 'A review on machinery diagnostics and prognostics implementing condition-based maintenance', *Mechanical Systems and Signal Processing*, 20(7), pp. 1483–1510. Available at: https://doi.org/10.1016/j.ymssp.2005.09.012.

Jia, Z., Xiao, Z. and Shi, Y. (2021) 'Remaining useful life prediction of equipment based on XGBoost', in *Proceedings of the 5th International Conference on Computer Science and Application Engineering (CSAE 2021)*. New York: ACM, article 59, pp. 1–6.

Kaufman, S., Rosset, S., Perlich, C. and Stitelman, O. (2012) 'Leakage in data mining: formulation, detection, and avoidance', *ACM Transactions on Knowledge Discovery from Data*, 6(4), article 15. Available at: https://doi.org/10.1145/2382577.2382579.

Khelif, R., Chebel-Morello, B., Malinowski, S., Laajili, E., Fnaiech, F. and Zerhouni, N. (2017) 'Direct remaining useful life estimation based on support vector regression', *IEEE Transactions on Industrial Electronics*, 64(3), pp. 2276–2285. Available at: https://doi.org/10.1109/TIE.2016.2623260.

Kuhn, M. and Johnson, K. (2013) *Applied predictive modeling*. New York: Springer.

LeCun, Y., Bengio, Y. and Hinton, G. (2015) 'Deep learning', *Nature*, 521(7553), pp. 436–444. Available at: https://doi.org/10.1038/nature14539.

Lei, Y., Li, N., Guo, L., Li, N., Yan, T. and Lin, J. (2018) 'Machinery health prognostics: a systematic review from data acquisition to RUL prediction', *Mechanical Systems and Signal Processing*, 104, pp. 799–834. Available at: https://doi.org/10.1016/j.ymssp.2017.11.016.

Li, X., Ding, Q. and Sun, J.-Q. (2018) 'Remaining useful life estimation in prognostics using deep convolution neural networks', *Reliability Engineering & System Safety*, 172, pp. 1–11. Available at: https://doi.org/10.1016/j.ress.2017.11.021.

Liu, Q., Zhang, Z., Guo, P., Wang, Y. and Liang, J. (2024) 'Enhancing aircraft engine remaining useful life prediction via multiscale deep transfer learning with limited data', *Journal of Computational Design and Engineering*, 11(1), pp. 343–355. Available at: https://doi.org/10.1093/jcde/qwae018.

MathWorks (n.d.) *What is Long Short-Term Memory (LSTM)?* Available at: https://www.mathworks.com/discovery/lstm.html (Accessed: 23 August 2026).

Mitchell, T.M. (1997) *Machine learning*. New York: McGraw-Hill.

Peel, L. (2008) 'Data driven prognostics using a Kalman filter ensemble of neural network models', in *2008 International Conference on Prognostics and Health Management*. Denver, CO, 6–9 October. Piscataway, NJ: IEEE, pp. 1–6. Available at: https://doi.org/10.1109/PHM.2008.4711504.

Ramasso, E. and Saxena, A. (2014) 'Performance benchmarking and analysis of prognostic methods for CMAPSS datasets', *International Journal of Prognostics and Health Management*, 5(2), pp. 1–15. Available at: https://doi.org/10.36001/ijphm.2014.v5i2.2236.

Rolls-Royce (2015) *The jet engine*. 5th edn. Chichester: John Wiley & Sons.

Saxena, A. and Goebel, K. (2008) *Turbofan engine degradation simulation data set* [Dataset]. NASA Ames Prognostics Data Repository. Moffett Field, CA: NASA Ames Research Center. Available at: https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository/ (Accessed: 5 August 2026).

Saxena, A., Goebel, K., Simon, D. and Eklund, N. (2008) 'Damage propagation modeling for aircraft engine run-to-failure simulation', in *2008 International Conference on Prognostics and Health Management*. Denver, CO, 6–9 October. Piscataway, NJ: IEEE, pp. 1–9. Available at: https://doi.org/10.1109/PHM.2008.4711414.

Si, X.-S., Wang, W., Hu, C.-H. and Zhou, D.-H. (2011) 'Remaining useful life estimation – a review on the statistical data driven approaches', *European Journal of Operational Research*, 213(1), pp. 1–14. Available at: https://doi.org/10.1016/j.ejor.2010.11.018.

Smola, A.J. and Schölkopf, B. (2004) 'A tutorial on support vector regression', *Statistics and Computing*, 14(3), pp. 199–222. Available at: https://doi.org/10.1023/B:STCO.0000035301.49549.88.

Sutton, R.S. and Barto, A.G. (2018) *Reinforcement learning: an introduction*. 2nd edn. Cambridge, MA: MIT Press.

Tippareddy, S. (2024) *Predicting remaining useful life of turbofan engines on CMAPSS and N-CMAPSS using deep recurrent neural networks*. Master's project. San Jose State University. Available at: https://doi.org/10.31979/etd.cyfe-z4sj.

Van Otten, N. (2024) 'Support Vector Regression (SVR) Simplified & How To Tutorial In Python', *Spot Intelligence*, 8 May. Available at: https://spotintelligence.com/2024/05/08/support-vector-regression-svr/ (Accessed: 23 August 2026).

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I. (2017) 'Attention is all you need', in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. Long Beach, CA, 4–9 December. Red Hook, NY: Curran Associates, pp. 5998–6008.

Wang, H., Li, D., Li, D., Liu, C., Yang, X. and Zhu, G. (2023) 'Remaining useful life prediction of aircraft turbofan engine based on Random Forest feature selection and multi-layer perceptron', *Applied Sciences*, 13(12), 7186. Available at: https://doi.org/10.3390/app13127186.

Wang, H.-K., Cheng, Y. and Song, K. (2021) 'Remaining useful life estimation of aircraft engines using a joint deep learning model based on TCNN and Transformer', *Computational Intelligence and Neuroscience*, 2021, article 5185938. Available at: https://doi.org/10.1155/2021/5185938.

Wheeler, R. (2009) *Turbofan2 Labelled* [Image]. Wikimedia Commons. Available at: https://commons.wikimedia.org/wiki/File:Turbofan2_Labelled.gif (Accessed: 10 August 2026).

Wu, F., Wu, Q., Tan, Y. and Xu, X. (2024) 'Remaining useful life prediction based on deep learning: a survey', *Sensors*, 24(11), 3454. Available at: https://doi.org/10.3390/s24113454.

Zheng, S., Ristovski, K., Farahat, A. and Gupta, C. (2017) 'Long short-term memory network for remaining useful life estimation', in *2017 IEEE International Conference on Prognostics and Health Management (ICPHM)*. Dallas, TX, 19–21 June. Piscataway, NJ: IEEE, pp. 88–95. Available at: https://doi.org/10.1109/ICPHM.2017.7998311.

Zhu, X. and Goldberg, A.B. (2009) *Introduction to semi-supervised learning*. San Rafael, CA: Morgan & Claypool.

APPENDICES

APPENDIX A
Data Set

APPENDIX B
Python Script

APPENDIX C
Project Files